

Repository Documentation

Project Structure

```
./
.gitignore
conftest.py
generate_pdf.py
pytest.ini
README.md
requirements.txt
docker/
  docker-compose.yml
pages/
  playwright/
    base_page_playwright.py
    header_page_playwright.py
    home_page_playwright.py
    login_page_playwright.py
    products_page_playwright.py
    registration_page_playwright.py
  selenium/
    base_page_selenium.py
    header_page_selenium.py
    home_page_selenium.py
    login_page_selenium.py
    products_page_selenium.py
    registration_page_selenium.py
    __init__.py
  playwright_tests/
    conftest.py
    pytest.ini
    test_adding_multiple_items_to_basket_playwright.py
    test_basket_playwright.py
    test_header_page_playwright.py
    test_home_page_playwright.py
    test_login_page_playwright.py
    test_registration_page_playwright.py
    test_search_and_sort_products_pw.py
  selenium_tests/
    conftest.py
    requirements
    test_adding_multiple_items_to_basket.py
    test_basket_selenium.py
    test_header_page_selenium.py
    test_home_page_selenium.py
    test_login_page_selenium.py
    test_registration_page_selenium.py
    test_search_and_sort_products_selenium.py
    __init__.py
  utils/
    config.py
    __init__.py
```

File: conftest.py

```
"""
Root conftest: register CLI options early so they work for any test path
(e.g. ``pytest selenium_tests/ --headless true``). Nested conftest
``pytest_addoption`` hooks can be registered too late for argument parsing.
"""

def pytest_addoption(parser):
    parser.addoption(
        "--remote",
        action="store",
        default="false",
        help="Selenium: use Remote WebDriver (Docker grid). true or false",
    )
    parser.addoption(
        "--headless",
        action="store",
        default="false",
        help="Selenium: run Chrome headless (new). true or false",
    )
```

File: generate_pdf.py

```
import os
from reportlab.platypus import SimpleDocTemplate, Paragraph, Preformatted, Spacer, PageBreak
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import cm

# === CONFIG ===
ROOT_DIR = "." # katalog repo (zmie■ je■li chcesz)
OUTPUT_FILE = "repo_documentation.pdf"

EXCLUDE_DIRS = {".git", "venv", "__pycache__", ".idea", ".vscode"}
INCLUDE_EXTENSIONS = {".py", ".txt", ".md", ".json", ".yaml", ".yml"}

# === INIT PDF ===
doc = SimpleDocTemplate(
    OUTPUT_FILE,
    pagesize=A4,
    rightMargin=2 * cm,
    leftMargin=2 * cm,
    topMargin=2 * cm,
    bottomMargin=2 * cm
)

styles = getSampleStyleSheet()
content = []

# === HELPERS ===

def add_title(text):
    content.append(Paragraph(text, styles["Title"]))
    content.append(Spacer(1, 12))

def add_heading(text):
    content.append(Paragraph(text, styles["Heading2"]))
    content.append(Spacer(1, 10))

def add_code(code_text):
    content.append(Preformatted(code_text, styles["Code"]))
    content.append(Spacer(1, 12))

def should_skip_dir(dirname):
    return dirname in EXCLUDE_DIRS

def build_tree_structure(root_dir):
    tree_lines = []
    for root, dirs, files in os.walk(root_dir):
        # filtr katalogów
        dirs[:] = [d for d in dirs if not should_skip_dir(d)]

        level = root.replace(root_dir, "").count(os.sep)
        indent = " " * level
        tree_lines.append(f"{indent}{os.path.basename(root)}/")

        sub_indent = " " * (level + 1)
        for f in files:
            tree_lines.append(f"{sub_indent}{f}")

    return "\n".join(tree_lines)

# === GENERATE ===

add_title("Repository Documentation")

# ■ STRUKTURA
add_heading("Project Structure")
tree = build_tree_structure(ROOT_DIR)
add_code(tree)

content.append(PageBreak())

# ■ PLIKI
for root, dirs, files in os.walk(ROOT_DIR):
```

```

dirs[:] = [d for d in dirs if not should_skip_dir(d)]

for file in files:
    filepath = os.path.join(root, file)

    # filtr rozszerze■
    if not any(file.endswith(ext) for ext in INCLUDE_EXTENSIONS):
        continue

    relative_path = os.path.relpath(filepath, ROOT_DIR)

    try:
        with open(filepath, "r", encoding="utf-8") as f:
            code = f.read()
    except Exception as e:
        code = f"ERROR READING FILE: {e}"

    add_heading(f"File: {relative_path}")
    add_code(code)

    # opcjonalnie podzia■ na strony (dla du■ych plików)
    content.append(PageBreak())

# === BUILD PDF ===
doc.build(content)

print(f"■ PDF generated: {OUTPUT_FILE}")

```

File: README.md

```
# Selenium vs Playwright – UI Test Automation

Postgraduate / portfolio project comparing **Selenium 4** and **Playwright** on the same **Telerik Kendra** demo app

---

## 1. Project description

### What it does

- Automates the same high-level user flows twice: once with **Selenium + Chrome**, once with **Playwright + Chromium**
- Targets a public demo e-commerce app so results are reproducible without a private backend.

### Problem it addresses

- Provides a **fair, side-by-side** comparison of frameworks for the same scenarios (login, registration, search, etc.)
- Collects evidence for **stability**, **runtime**, and **maintainability** (structure, waits, flakiness, etc.)

### Technologies (from `requirements.txt` and code)

| Area | Stack |
|-----|-----|
| Language | Python 3 |
| Test runner | **pytest** 9.x |
| Selenium | **selenium** 4.x, **WebDriverWait**, Chrome (local or Grid) |
| Playwright | **playwright**, **pytest-playwright** |
| Parallel runs | **pytest-xdist** |
| Reporting | **allure-pytest** |
| Other | **requests**, **mysql-connector-python** (deps in file; DB not required for UI tests) |

---

## 2. Project structure

```
.
├── conftest.py # Root: registers --remote and --headless for Selenium (early pytest hook)
├── pytest.ini # Global pytest defaults, testpaths, markers
├── requirements.txt # Python dependencies (pin versions)
├── README.md # This file
├── docker/
│ └── docker-compose.yml # Selenium Grid hub + Chrome node, Playwright image, Allure services
├── selenium_tests/ # Selenium test package
│ ├── conftest.py # Session driver, reset_browser_state, page fixtures, Allure screenshots
│ ├── test_*.py # Test modules (login, home, header, basket, registration, search/sort, etc.)
│ └── ...
├── playwright_tests/ # Playwright test package
│ ├── conftest.py # Page fixtures, browser_context_args, launch args, Allure on failure
│ ├── pytest.ini # pythonpath for this subtree (used when running from this folder)
│ └── test_*.py # Mirror flows to Selenium (naming includes minor typos in filenames)
├── pages/
│ ├── selenium/ # Selenium Page Objects (inherit BasePage)
│ │ ├── base_page_selenium.py
│ │ ├── home_page_selenium.py
│ │ ├── login_page_selenium.py
│ │ ├── header_page_selenium.py
│ │ ├── products_page_selenium.py
│ │ └── registration_page_selenium.py
│ └── playwright/ # Playwright Page Objects
│ ├── base_page_playwright.py
│ ├── home_page_playwright.py
│ ├── login_page_playwright.py
│ ├── header_page_playwright.py
│ ├── products_page_playwright.py
│ └── registration_page_playwright.py
├── utils/
│ └── config.py # Shared constants (BASE_URL, demo credentials, default timeout)
└── reports/ # Optional: Allure raw results / generated HTML (create locally; see DOCS)
```

| Path | Role |
```

```
|-----|-----|
| **`conftest.py` (root)** | Registers **`--remote`** and **`--headless`** so `pytest selenium_tests/`
| **`pytest.ini`** | `pythonpath = .`, discovers `selenium_tests` and `playwright_tests`, `--durations`,
| **`selenium_tests/conftest.py`** | Chrome `Options`, session-scoped `driver`, light vs heavy browser
| **`playwright_tests/conftest.py`** | `page`-based fixtures, `base_url`, headed launch defaults, Allure
| **`pages/selenium/`** | Encapsulates locators + waits + actions for Selenium. |
| **`pages/playwright/`** | Same idea for Playwright (`page` API). |
| **`utils/config.py`** | Central URL and test user constants (credentials also documented below). |
```

3. How to run tests

Use a virtual environment (recommended).

Install dependencies

```
```bash
python -m venv .env
Windows:
.env\Scripts\activate
Linux/macOS:
source .env/bin/activate

pip install -r requirements.txt
```
```

Playwright browsers (required for Playwright tests)

```
```bash
python -m playwright install chromium
```
```

Selenium – local Chrome

```
```bash
python -m pytest selenium_tests/ --remote false -v
```
```

Selenium – headless Chrome

Uses Chrome flag `--headless=new` (requires root `conftest.py` so the option is registered).

```
```bash
python -m pytest selenium_tests/ --remote false --headless true -v
```
```

Selenium – Docker Grid (`localhost:4444`)

Start Grid first (see [Docker](#docker-optional)).

```
```bash
python -m pytest selenium_tests/ --remote true -v
```
```

Selenium – parallel (`pytest-xdist`)

Each worker gets its own browser session. Ensure enough machine resources and, for `--remote true`, enough

```
```bash
python -m pytest selenium_tests/ -n auto -q
```
```

Combine with headless:

```
```bash
python -m pytest selenium_tests/ --headless true -n auto -q
```
```

Playwright

From repo root (uses root `pytest.ini`):

```
```bash
python -m pytest playwright_tests/ -v
```
```

Or from `playwright\_tests/` (uses local `pytest.ini` + `pythonpath`):

```
```bash
cd playwright_tests
```
```

```
python -m pytest -v
cd ..
```
```

```
Run both suites (default discovery)
```

```
```bash
python -m pytest -v
```
```

```
Slowest tests (built into `pytest.ini`)
```

```
```bash
python -m pytest selenium_tests/ --durations=15
```
```

```
Allure results
```

```
```bash
python -m pytest selenium_tests/ -v --alluredir=reports/allure-results
python -m pytest playwright_tests/ -v --alluredir=reports/allure-results
allure serve reports/allure-results
```
```

Install [Allure Commandline](<https://github.com/allure-framework/allure2>) and Java if `allure` is not on

---

#### ## 4. System requirements

| Requirement    | Notes                                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------------------|
| **Python**     | 3.10+ recommended (project tested with 3.12/3.13 in CI-like environments).                                  |
| **pip**        | For `requirements.txt`.                                                                                     |
| **Chrome**     | Selenium local runs use **Google Chrome**; match **ChromeDriver** (Selenium Manager in 4.                   |
| **Playwright** | `playwright install chromium` after `pip install`.                                                          |
| **Docker**     | (optional)   For Grid + Allure stack (`docker/docker-compose.yml`).                                         |
| **Network**    | Tests hit <a href="https://demos.telerik.com">**https://demos.telerik.com**</a> – stable internet required. |
| **Java**       | Only if you generate/serve Allure reports via CLI.                                                          |

---

#### ## 5. How the framework works

##### ### Page Object Model (POM)

- Tests stay thin; **locators and waits** live under `pages/selenium/` and `pages/playwright/`.
- **Selenium**: classes inherit `BasePage` (`wait_for_visible`, `safe_click`, `open`, etc.).
- **Playwright**: page classes wrap `page` (`goto`, `locator`, `expect` patterns in each file).

##### ### Fixtures

| Framework      | Key fixtures                                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| **Selenium**   | <code>driver</code> (session), <code>reset_browser_state</code> (autouse, per test), <code>*_page_selenium</code> factories           |
| **Playwright** | <code>page</code> (from <code>pytest-playwright</code> ), <code>browser_context_args</code> , <code>browser_type_launch_args</code> , |

##### ### Setup / teardown (Selenium)

- **Session-scoped `driver`**: one browser per pytest worker.
- **Heavy reset once** after driver start: navigate to eShop origin + CDP `Storage.clearDataForOrigin` +
- **Light reset before each test**: `get` eShop entry URL + delete cookies + clear `localStorage` / `sessionStorage`

##### ### Base pages

- `pages/selenium/base_page_selenium.py` – shared waits, clicks, navigation helpers.
- `pages/playwright/base_page_playwright.py` – shared Playwright helpers.

##### ### Utils

- `utils/config.py` – `BASE_URL`, demo `USER_EMAIL` / `USER_PASSWORD`, `DEFAULT_TIMEOUT`, optional `SELECTOR`

##### ### Demo credentials (public demo user)

```
```text
Email:    jaxons.danniels@company.com
Password: User1234
```
```

---

#### ## 6. Adding a new test

1. **Choose stack**: ``selenium_tests/test_<feature>selenium.py`` or ``playwright_tests/test_<feature>playwright.py``.
2. **Import** Allure markers if needed (``@allure.suite``, etc., match existing style).
3. **Request fixtures**: e.g. ``home_page_selenium``, ``login_page_selenium``, ``product_page_selenium`` or ``page``.
4. **Call page methods** in the same order as the user story; keep assertions readable (``assert`` + message).
5. **Run**: ``python -m pytest path/to/test_file.py -v``.
6. **Optional**: add ``-p allure_pytest --alluredir=reports/allure-results`` for reporting.

Do not rely on another test having run first; use fixtures and POM methods to reach a known state.

---

## ## 7. Adding a new Page Object

### ### Selenium

1. Create ``pages/selenium/<name>_page_selenium.py``.
2. Subclass ``BasePage``; store **locators** as class attributes (``By...``, ``"..."``).
3. Implement methods using ``self.wait_for_``, ``self.click``, ``self.open``, etc.
4. Register a **pytest fixture** in ``selenium_tests/conftest.py`` that returns ``YourPage(self.driver)``.

### ### Playwright

1. Create ``pages/playwright/<name>_page_playwright.py``.
2. Accept ``page`` in ``__init__`` (see existing pages).
3. Use ``self.page.goto``, locators, and ``expect`` where appropriate.
4. Add a **fixture** in ``playwright_tests/conftest.py``.

Keep framework-specific code in the correct ``pages/`` subtree so imports stay clear.

---

## ## 8. CI/CD

There is **no** ``.github/workflows/`` (or similar) in this repository at the time of writing. For CI, a t

1. ``pip install -r requirements.txt``
2. ``playwright install chromium --with-deps`` (Linux agents)
3. Install Chrome + run Selenium with ``--headless true`` or use a Selenium service container
4. ``pytest`` with optional ``-n auto`` and ``--alluredir``, then upload Allure artifacts

---

## ## 9. Known issues & troubleshooting

| Symptom                                                                  | What to check                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>**unrecognized arguments: --headless**</code>                      | Ensure <code>**`conftest.py`</code> exists at repository root** (register <code>**`Selenium Grid`</code> connection errors**   <code>`docker compose up`</code> in <code>`docker/`</code> , then <code>`--remote true`</code> . Hub: <code>`ht</code> |
| <code>**pytest.mark.flaky` warning**</code>                              | Register the marker in <code>`pytest.ini`</code> or install <code>`pytest-rerunfailures`</code>                                                                                                                                                       |
| <code>**Playwright `headless: False`**</code>                            | Browsers open visibly by design in <code>`playwright_tests/conftest.py`</code> ;                                                                                                                                                                      |
| <code>**Duplicate `browser_context_args` in Playwright conftest**</code> | File currently defines the name twice –                                                                                                                                                                                                               |
| <code>**Flaky demo**</code>                                              | <code>`demos.telerik.com`</code> is shared and rate-limited; retries or longer waits may be needed                                                                                                                                                    |
| <code>**Allure `command not found`**</code>                              | Install Allure CLI + <code>**`JAVA_HOME`**</code> .                                                                                                                                                                                                   |

---

## ## 10. Performance & stability tips (implemented or recommended)

- **Parallel Selenium**: ``pytest -n auto`` – each worker has its own session ``driver`` (``selenium_tests/co`
- **Avoid implicit waits** with explicit ``WebDriverWait`` – Selenium uses ``implicitly_wait(0)``.
- **Prefer explicit conditions** (``element_to_be_clickable``, ``visibility_of_element_located``) over fixed
- **Session vs per-test work**: heavy CDP storage/cache clear runs **once per session**; lighter cookie
- **Headless for throughput**: ``--headless true`` for local/CI when you do not need a visible window.
- **Stable selectors**: favour ``ID``, ``data-``, or scoped CSS; XPath is used where the demo DOM requires
- **Direct URLs** where possible inside POM (``open``, ``_get_url_fast``) to skip redundant clicks when the

---

### ## Docker (optional)

From repository root:

```
``bash
cd docker
docker compose up -d
``
```

Typical ports (see ``docker/docker-compose.yml``):

| Service | Port  | Purpose |
|---------|-------|---------|
| -----   | ----- | -----   |



```
Selenium Hub	4444	Grid / WebDriver endpoint
Chrome node	7900	noVNC preview
Allure API / UI	5050 / 5252	Report generation & dashboard
```

Stop:

```
```bash
cd docker
docker compose down
```
```

---

## Changelog vs previous README (summary)

```
Topic	Old README	Current state
Headless Selenium	Stated headless was ignored	**`--headless true`** is supported (Chrome `--headless` / `--remote` parsing).
Root `confest.py`	Not documented	Documented; required for `--headless` / `--remote` parsing.
pytest-xdist	Not mentioned	In `requirements.txt`; documented `-n auto`.
Selenium reset	Implied per-test heavy work	**Heavy CDP once per session**, light reset per test.
Structure	High-level only	Full tree + table for `pages/`, `utils/`, `docker/`.
Playwright	Basic run	Root vs `playwright_tests/` run, `install chromium`.
CI/CD	Not clearly stated	**No workflows in repo**; generic CI guidance added.
Troubleshooting	Partial	Expanded (headless, Grid, flaky marker, Playwright confest note).
```

---

## Future improvements (ideas)

- Add \*\*GitHub Actions\*\* workflow matrix (Selenium headed/headless + Playwright) with cached browsers.
- Resolve \*\*duplicate `browser\_context\_args`\*\* fixtures in `playwright\_tests/confest.py`.
- Optional \*\*`Makefile`\*\* or \*\*`nox`\*\* tasks for `lint`, `test-selenium`, `test-pw`, `allure`.
- \*\*Pre-commit\*\* (e.g. `ruff`) for consistent Python style.
- \*\*Environment file\*\* (`.env.example`) if secrets or base URLs move out of code.

---

## License / thesis

Use and cite this repository according to your institution's rules for academic work. The automated target

## **File: requirements.txt**

```
ERROR READING FILE: 'utf-8' codec can't decode byte 0xff in position 0: invalid start byte
```

## File: docker\docker-compose.yml

```
version: "3.9"

services:
 # =====
 # SELENIUM GRID - HUB
 # =====
 selenium-hub:
 image: selenium/hub:4.18.1
 container_name: selenium-hub
 ports:
 - "4444:4444" # Grid UI + test endpoint
 networks:
 - testnet

 # =====
 # SELENIUM GRID - CHROME NODE
 # =====
 chrome:
 image: selenium/node-chrome:4.18.1
 container_name: chrome
 shm_size: 2gb # Prevents Chrome "Aw, Snap!" crashes
 depends_on:
 - selenium-hub
 environment:
 - SE_EVENT_BUS_HOST=selenium-hub
 - SE_EVENT_BUS_PUBLISH_PORT=4442
 - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
 - SE_NODE_MAX_SESSIONS=2
 - START_XVFB=true
 - VNC_NO_PASSWORD=1
 ports:
 - "7900:7900" # noVNC viewer port
 networks:
 - testnet

 # =====
 # PLAYWRIGHT (official image)
 # =====
 playwright:
 image: mcr.microsoft.com/playwright/python:v1.42.0-focal
 container_name: playwright
 working_dir: /tests
 volumes:
 - ../playwright_tests:/tests
 networks:
 - testnet
 command: ["sleep", "infinity"]

 # =====
 # ALLURE REPORT - GENERATOR
 # =====
 allure-generator:
 image: frankescobar/allure-docker-service
 container_name: allure-generator
 ports:
 - "5050:5050"
 environment:
 - CHECK_RESULTS_EVERY_SECONDS=15
 - KEEP_HISTORY=1
 volumes:
 - ../reports/allure-results:/app/allure-results
 - ../reports/allure-report:/app/allure-report
 networks:
 - testnet

 # =====
 # ALLURE REPORT - UI
 # =====
 allure-ui:
 image: frankescobar/allure-docker-service-ui
 container_name: allure-ui
 ports:
 - "5252:5252"
 environment:
 - ALLURE_DOCKER_PUBLIC_API_URL=http://localhost:5050
 depends_on:
 - allure-generator
```

```
 networks:
 - testnet

=====
Shared network
=====
networks:
 testnet:
 driver: bridge
```

## File: pages\playwright\base\_page\_playwright.py

```
import re
import time
import allure
from playwright.sync_api import Page, expect

class BasePagePw:
 def __init__(self, page: Page, timeout_ms: int = 4000):
 self.page = page
 self.timeout_ms = timeout_ms
 self._log("STEP", "Initializing page object and viewport 1920x1080")
 self.page.set_viewport_size({"width": 1920, "height": 1080})

 def _log(self, level: str, message: str):
 print(f"[POM][{level}] {message}")

 def log_step(self, message: str):
 self._log("STEP", message)

 def log_info(self, message: str):
 self._log("INFO", message)

 def log_assert(self, message: str):
 self._log("ASSERT", message)

 def log_done(self, message: str):
 self._log("DONE", message)

 def log_warn(self, message: str):
 self._log("WARN", message)

 def log_error(self, message: str):
 self._log("ERROR", message)

 def open(self, url: str):
 self._log("STEP", f"Navigating to URL: {url}")
 self.safe_goto(url, wait_until="domcontentloaded")

 def safe_goto(self, url: str, wait_until: str = "domcontentloaded", retries: int = 3):
 last_exception = None
 for attempt in range(retries):
 try:
 self.page.goto(url, wait_until=wait_until)
 return
 except Exception as exc: # Playwright Timeout/Error classes are runtime-provided.
 last_exception = exc
 if attempt < retries - 1:
 self.log_warn(
 f"Navigation failed (attempt {attempt + 1}/{retries}) for '{url}': {exc}. Retrying..."
)
 time.sleep(0.4)
 continue
 self.log_error(f"Navigation failed after {retries} attempts: {url}")
 raise last_exception

 def click(self, selector: str):
 self._log("STEP", f"Clicking element: {selector}")
 locator = self.page.locator(selector)
 expect(locator).to_be_visible(timeout=self.timeout_ms)
 locator.click()

 def js_click(self, selector: str):
 self._log("STEP", f"Executing JS click on element: {selector}")
 locator = self.page.locator(selector)
 # JS click is used as fallback also for covered/hidden nodes.
 expect(locator).to_have_count(1, timeout=self.timeout_ms)
 locator.evaluate("el => el.click()")

 def type(self, selector: str, text: str):
 self._log("STEP", f"Typing text into field: {selector}")
 locator = self.page.locator(selector)
 expect(locator).to_be_visible(timeout=self.timeout_ms)
 locator.fill(text)

 def get_title(self):
 title = self.page.title()
 self._log("INFO", f"Retrieved page title: '{title}'")
 return title
```

```

def wait_for_visible(self, selector: str):
 locator = self.page.locator(selector)
 expect(locator).to_be_visible(timeout=self.timeout_ms)
 return locator

def wait_for_url(self, expected_url_fragment: str):
 # Use regex "contains" semantics to avoid fragile glob-base_url interactions.
 expect(self.page).to_have_url(
 re.compile(re.escape(expected_url_fragment)),
 timeout=self.timeout_ms,
)

def screenshot(self, name="screenshot"):
 self._log("INFO", f"Taking screenshot: {name}")
 allure.attach(
 self.page.screenshot(),
 name=name,
 attachment_type=allure.attachment_type.PNG
)

```

## File: pages\playwright\header\_page\_playwright.py

```
import re
from playwright.sync_api import Page, expect

class HeaderPagePw:
 def __init__(self, page: Page):
 self.page = page
 # Reuse the same locators as Selenium for fair framework comparison.
 self.SHOPPING_CART_BTN = "//a[contains(@href, 'ShoppingCart')]"
 self.CATEGORIES_MENU = "//span[normalize-space()='Categories']"
 self.ACCESSORIES_LINK = "//a[contains(@class, 'k-menu-link') and normalize-space()='Accessories']"
 self.BIKES_LINK = "//a[contains(@class, 'k-menu-link') and normalize-space()='Bikes']"
 self.CLOTHES_LINK = "//a[contains(@class, 'k-menu-link') and normalize-space()='Clothes']"
 self.COMPONENTS_LINK = "//a[contains(@class, 'k-menu-link') and normalize-space()='Components']"

 def click_cart_pw(self):
 self.page.locator(self.SHOPPING_CART_BTN).first.click()

 def open_home_page_pw(self):
 self.page.goto("https://demos.telerik.com/kendo-ui/eshop", wait_until="domcontentloaded")

 def get_url_pw(self):
 return self.page.url

 def assert_url_contains_pw(self, expected_fragment: str):
 expect(self.page).to_have_url(re.compile(re.escape(expected_fragment)))

 def validate_header_navigation_pw(self):
 checks = [
 ("ShoppingCart", self.click_cart_pw),
 ("eshop", self.open_home_page_pw),
 ("Home/Accessories", self.open_accessories_category_pw),
 ("Home/Bikes", self.open_bikes_category_pw),
 ("Home/Clothing", self.open_clothes_category_pw),
 ("Home/Components", self.open_components_category_pw),
]

 for expected_fragment, action in checks:
 self.open_home_page_pw()
 action()
 self.assert_url_contains_pw(expected_fragment)

 # Category hover helper methods
 def _hover_menu_pw(self):
 # Select first matching element to avoid strict-mode ambiguity.
 self.page.locator(self.CATEGORIES_MENU).first.hover()

 def open_accessories_category_pw(self):
 self._hover_menu_pw()
 self.page.locator(self.ACCESSORIES_LINK).click()

 def open_bikes_category_pw(self):
 self._hover_menu_pw()
 self.page.locator(self.BIKES_LINK).click()

 def open_clothes_category_pw(self):
 self._hover_menu_pw()
 self.page.locator(self.CLOTHES_LINK).click()

 def open_components_category_pw(self):
 self._hover_menu_pw()
 self.page.locator(self.COMPONENTS_LINK).click()
```

## File: pages\playwright\home\_page\_playwright.py

```
from pages.playwright.base_page_playwright import BasePagePw

class HomePagePw(BasePagePw):
 def __init__(self, page):
 super().__init__(page)

 # --- URL's ---
 HOME_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop"
 REGISTRATION_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop/Account/Register"
 LOGIN_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop/Account/Login"
 CATEGORY_BIKES_URL = "https://demos.telerik.com/kendo-ui/eshop/Home/Bikes"

 # --- LOKATORY ---
 LOGOUT_BUTTON = "//a[contains(@href, '/Account/Logout')]"
 MAIN_LOGIN_LINK = "//button[@type='submit' and contains(., 'Login')]"

 """Navigation methods"""

 def open_home_page_pw(self):
 self.log_step(f"Navigate to home page: {self.HOME_PAGE_URL}")
 self.open(self.HOME_PAGE_URL)
 return self

 def open_login_page_pw(self):
 self.log_step("Navigate to login page")
 self.open(self.LOGIN_PAGE_URL)
 return self

 def open_register_page_pw(self):
 self.log_step("Navigate to registration page")
 self.open(self.REGISTRATION_PAGE_URL)
 return self

 def click_logout_button_pw(self):
 self.log_step("Click logout action via JS")
 self.js_click(self.LOGOUT_BUTTON)
 return self

 """Verification methods"""

 def is_logout_button_displayed_pw(self):
 self.log_step("Check whether logout action is available")
 is_visible = self.page.locator(self.LOGOUT_BUTTON).count() > 0
 if is_visible:
 self.log_done("Logout action found")
 else:
 self.log_warn("Logout action was not found")
 return is_visible
```



## File: pages\playwright\login\_page\_playwright.py

```
from playwright.sync_api import Page, expect
from pages.playwright.base_page_playwright import BasePagePw

class LoginPagePw(BasePagePw):
 def __init__(self, page: Page):
 super().__init__(page)
 self.page = page

 ADMIN_TEST_USER_EMAIL = "jaxons.danniels@company.com"
 ADMIN_TEST_USER_PASS = "User1234"

 # Locators
 LOGIN_FIELD = "#Email"
 PASSWORD_FIELD = "#Password"
 LOGIN_BUTTON = ".k-form-submit"
 AUTHENTICATED_FAVORITES_LINK = "//a[contains(@href, '/Account/Favorites')]"

 def login_as_admin_pw(self):
 self.log_step(f"Filling email field for user: {self.ADMIN_TEST_USER_EMAIL}")
 self.page.fill(self.LOGIN_FIELD, self.ADMIN_TEST_USER_EMAIL)

 self.log_step("Filling password field")
 self.page.fill(self.PASSWORD_FIELD, self.ADMIN_TEST_USER_PASS)

 self.log_step("Clicking login button")
 self.page.click(self.LOGIN_BUTTON)

 self.log_step("Waiting for post-login authenticated state")
 self.wait_for_url("/eshop")
 self.wait_for_visible(self.AUTHENTICATED_FAVORITES_LINK)
 self.log_done("Login completed successfully")

 return self

 def is_logged_in_pw(self):
 return self.page.locator(self.AUTHENTICATED_FAVORITES_LINK).first.is_visible()

 def expect_logged_in_pw(self):
 expect(self.page.locator(self.AUTHENTICATED_FAVORITES_LINK).first).to_be_visible(timeout=self.t)
```

## File: pages\playwright\products\_page\_playwright.py

```
from playwright.sync_api import expect
from pages.playwright.base_page_playwright import BasePagePw

class ProductPagePw(BasePagePw):
 def __init__(self, page):
 super().__init__(page)

 """LOCATORS"""
 BIKE_MAIN_LINK = "https://demos.telerik.com/kendo-ui/eshop/Home/Bikes"
 BIKE_CATEGORY_TITLES = "//div[@class='category-heading']"
 MOUNTAIN_BIKES = "//a[contains(@href, 'subCategory=Mountain Bikes')]"
 PRODUCT_TITLES = "//div[@class='k-card-title']"
 ROAD_BIKES = "//a[contains(@href, 'subCategory=Road Bikes')]"
 TOURING_BIKES = "//a[contains(@href, 'subCategory=Touring Bikes')]"

 ALL_BIKES_FILTER = "//label[contains(text(), 'All')]"
 DISCOUNTED_BIKES_FILTER = "//label[contains(text(), 'Discounted items only')]"

 # Pager and cards
 PAGER_INFO = "//span[contains(@class, 'k-pager-info')]"
 PRODUCT_CARDS = "//div[contains(@class, 'k-card') and not(contains(@class, 'k-card-list'))]"
 DISCOUNT_BADGE = "//span[@class='discount-pct']"

 # Sorting
 SORT_TRIGGER = "//span[contains(@class, 'k-input-value-text')]"
 SORT_OPTION_PRICE_DESC = "text=Price - High to Low"
 SORT_OPTION_PRICE_ASC = "text=Price - Low to High"
 CART_ICON = "//a[contains(@href, 'Cart')]"
 REMOVE_BTN = "//p[text()='Remove']"
 DISCOUNTED_FILTER = "//label[contains(text(), 'Discounted items only')]"
 PRICE_LABELS = "//div[@class='card-price']"
 ADD_TO_CART_BTN = "(//button[contains(@class, 'add-to-cart') or contains(text(), 'Add to Cart')])"
 CART_TOTAL_PRICE = "subTotalValue"
 ALL_ADD_BUTTONS = "add-to-cart"
 CART_LINE_PRICES_PW = "//td[contains(@class, 'final-price')]"

 """ACTIONS AND ASSERTIONS"""

 def open_bikes_main_link_pw(self):
 self.log_step(f"Navigating to bikes landing page: {self.BIKE_MAIN_LINK}")
 self.open(self.BIKE_MAIN_LINK)
 return self

 def get_bike_category_titles_pw(self):
 self.log_step("Collecting bike category titles")
 return self.page.locator(self.BIKE_CATEGORY_TITLES).all_inner_texts()

 def open_mountain_bikes_pw(self):
 self.log_step("Opening category: Mountain Bikes")
 # 1. Wait until the link exists in the DOM
 selector = self.page.locator(self.MOUNTAIN_BIKES).first
 selector.wait_for(state="attached", timeout=10000)

 # 2. force=True skips Playwright actionability checks (e.g. obscured elements)
 selector.click(force=True)

 self.page.wait_for_load_state("networkidle")
 return self

 def click_filter_all_pw(self):
 self.log_step("Applying filter: All")
 self.page.locator(self.ALL_BIKES_FILTER).click()
 self.page.wait_for_timeout(1000)
 return self

 def click_filter_discounted_pw(self):
 self.log_step("Applying filter: Discounted items only")
 self.page.locator(self.DISCOUNTED_BIKES_FILTER).click()
 self.page.wait_for_timeout(1000)
 return self

 def get_total_count_from_pager_pw(self):
 self.log_step("Reading product count from pager")
 self.page.wait_for_selector(self.PAGER_INFO, state="visible", timeout=5000)
```

```

text = self.page.locator(self.PAGER_INFO).inner_text()
self.log_info(f"Pager text: '{text}'")
count = int(text.split("of")[-1].strip().split(" ")[0])
self.log_info(f"Parsed pager count: {count}")
return count

def get_cart_line_item_prices_pw(self):
 """Read unit prices from the cart table."""
 self.log_info("[POM] Reading line prices from the cart table...")
 price_locators = self.page.locator(self.CART_LINE_PRICES_PW)

 # Parse text, strip currency, convert to float
 raw_texts = price_locators.all_inner_texts()
 prices = []
 for text in raw_texts:
 clean_price = float(text.replace('$', '').replace(',', '').strip())
 prices.append(clean_price)

 self.log_info(f"[POM] Cart line prices: {prices}")
 return prices

def assert_expected_bike_categories_pw(self, expected_titles=None):
 titles = self.get_bike_category_titles_pw()
 self.log_info(f"Bike categories found: {titles}")
 expected_titles = expected_titles or ["Mountain Bikes", "Road Bikes", "Touring Bikes"]
 for expected in expected_titles:
 assert expected in titles, f"Expected category '{expected}' in {titles}"

def assert_products_available_for_all_filter_pw(self):
 self.click_filter_all_pw()
 all_count = self.get_total_count_from_pager_pw()
 assert all_count > 0, f"Expected products for 'All' filter, got count={all_count}"
 return all_count

def assert_discount_filter_consistency_pw(self, expected_all_count=None):
 self.click_filter_discounted_pw()
 actual_discounted_pager = self.get_total_count_from_pager_pw()
 actual_badges = self.count_badges_pw()
 visible_cards = self.count_visible_bikes_pw()
 if expected_all_count is not None:
 assert actual_discounted_pager <= expected_all_count, (
 f"Discounted pager count ({actual_discounted_pager}) should not exceed all-items count '{expected_all_count}'."
)
 assert actual_discounted_pager > 0, "Discount filter should return at least one product."
 assert actual_badges == actual_discounted_pager, (
 f"Discount badge count ({actual_badges}) should match pager count ({actual_discounted_pager})"
)
 assert visible_cards == actual_discounted_pager, (
 f"Visible discounted cards ({visible_cards}) should match pager count ({actual_discounted_pager})"
)

def assert_filter_counts_consistent_pw(self):
 all_count = self.assert_products_available_for_all_filter_pw()
 self.assert_discount_filter_consistency_pw(expected_all_count=all_count)

def count_badges_pw(self):
 self.page.wait_for_selector(self.DISCOUNT_BADGE, state="visible", timeout=5000)
 count = self.page.locator(self.DISCOUNT_BADGE).count()
 self.log_info(f"Discount badge count: {count}")
 return count

def debug_page_content_pw(self):
 all_badges = self.page.locator("//span[contains(@class, 'k-badge')]").all_inner_texts()
 self.log_info(f"Badges visible on page: {all_badges}")

def count_visible_bikes_pw(self):
 count = self.page.locator(self.PRODUCT_CARDS).filter(has=self.page.locator(".discount-pct")).count()
 self.log_info(f"Visible discounted cards count: {count}")
 return count

def clear_cart_pw(self):
 self.log_step("Starting BRUTE FORCE cart cleanup")

 # Accept confirm dialogs during removal
 self.page.on("dialog", lambda dialog: dialog.accept())

 while True:
 self.go_to_cart_pw()
 # Wait for the cart page to settle
 self.page.wait_for_load_state("networkidle")

```

```

 remove_buttons = self.page.locator(f"xpath={self.REMOVE_BTN}")
 count = remove_buttons.count()

 if count == 0:
 break # Cart is empty

 self.log_info(f"Items still in cart: {count}. Removing one and refreshing...")

 # Click Remove on the first line
 remove_buttons.first.click()

 # Short wait then hard reload so the grid updates
 self.page.wait_for_timeout(1000)
 self.page.reload()

 self.log_done("Cart is finally empty.")
 self.open_bikes_main_link_pw()
 return self

def go_to_cart_pw(self):
 self.log_step("Opening shopping cart")
 self.page.click(f"xpath={self.CART_ICON}")
 return self

def add_first_product_pw(self):
 self.log_step("Adding first product to cart")
 # Use .first to explicitly target the first matching element.
 self.page.locator(f"xpath={self.ADD_TO_CART_BTN}").first.click()
 return self

def get_all_prices_pw(self):
 self.log_step("Collecting product prices")
 locators = self.page.locator(f"xpath={self.PRICE_LABELS}")
 return [float(el.inner_text().replace('$', '').replace(',', '')) for el in locators.all()]

def get_all_names_pw(self):
 self.log_step("Collecting product names")
 locators = self.page.locator(f"xpath={self.PRODUCT_TITLES}")
 return [el.inner_text().strip() for el in locators.all()]

def add_first_product_to_cart_and_verify_pw(self):
 product_names = self.get_all_names_pw()
 first_product_name = product_names[0]
 self.log_info(f"Selected product: '{first_product_name}'")

 self.log_step("Adding selected product to cart")
 self.add_first_product_pw()

 self.log_step("Opening cart for verification")
 self.go_to_cart_pw()

 self.log_assert(f"Checking whether '{first_product_name}' is visible in cart")
 cart_item_locator = self.page.locator(f"//*[contains(text(), '{first_product_name}')]")

 expect(cart_item_locator.first).to_be_visible()
 self.log_done("Product element found in cart")

 expect(cart_item_locator.first).to_contain_text(first_product_name)
 self.log_done("Cart item name matches selected product")
 return self

def all_sorting_options_pw(self):
 self.log_step("Starting sorting sequence")

 # Open sorting dropdown.
 self.page.locator(self.SORT_TRIGGER).click()

 # Choose option: Price - High to Low.
 self.page.locator("li[role='option']:has-text('Price - High to Low')").click()

 self.log_step("Applied sort: Price - High to Low")
 self.page.wait_for_timeout(1000)

 # Open dropdown again.
 self.page.locator(self.SORT_TRIGGER).click()

 # Choose option: Price - Low to High.
 self.page.locator("li[role='option']:has-text('Price - Low to High')").click()

 self.log_step("Applied sort: Price - Low to High")

```

```

self.page.wait_for_timeout(1000)

self.log_done("Sorting sequence finished")

def add_multiple_products_and_verify_total_pw(self, count=5):
 """Adds N products to cart and validates that subtotal matches line items and original prices."""
 self.log_step(f"Starting total-price validation for {count} products")

 # 1. Capture prices from the shop floor
 all_prices = self.get_all_prices_pw()
 if len(all_prices) < count:
 raise Exception(f"Found only {len(all_prices)} products, need {count}")

 selected_prices = all_prices[:count]
 expected_total = sum(selected_prices)
 self.log_info(f"Expected total based on shop cards: ${expected_total:.2f}")

 # 2. Add products to cart
 add_buttons = self.page.locator(self.ADD_TO_CART_BTN)
 for i in range(count):
 self.log_step(f"Adding product {i + 1}/{count}")
 add_buttons.nth(i).click()
 # Small delay to allow Kendo UI animations to trigger
 self.page.wait_for_timeout(300)

 # 3. Go to Cart
 self.go_to_cart_pw()

 # 4. Read Subtotal from the UI
 self.log_step("Reading cart subtotal from the UI...")
 total_locator = self.page.locator(f"#{self.CART_TOTAL_PRICE}")

 expect(total_locator).to_contain_text("$", timeout=5000)

 total_text = total_locator.inner_text()
 actual_total = float(total_text.replace('$', '').replace(',', '').strip())

 # 5. Read individual line prices from the table
 cart_line_prices = self.get_cart_line_item_prices_pw()
 cart_line_sum = sum(cart_line_prices)

 # Logging for Allure/Console
 self.log_assert(f"Expected Sum (Cards): ${expected_total:.2f}")
 self.log_assert(f"Cart Line Sum: ${cart_line_sum:.2f}")
 self.log_assert(f"Cart Subtotal: ${actual_total:.2f}")

 # Final Validations
 assert round(cart_line_sum, 2) == round(actual_total, 2), \
 f"Line items sum ({cart_line_sum}) does not match subtotal ({actual_total})"

 assert round(actual_total, 2) == round(expected_total, 2), \
 f"Subtotal ({actual_total}) does not match original card prices ({expected_total})"

 self.log_done("Cart totals and line items verified successfully")
 return self

```

## File: pages\playwright\registration\_page\_playwright.py

```
from playwright.sync_api import Page, expect
from pages.playwright.base_page_playwright import BasePagePw

class RegistrationPagePw(BasePagePw):
 def __init__(self, page: Page):
 super().__init__(page)

 # --- Validation summary errors ---
 self.EMPTY_FIRST_AND_LAST_NAME_VALIDATION_SUMMARY_LIST = "//li[contains(., 'First and Last name is required')]"
 self.EMPTY_EMAIL_ERROR_VALIDATION_SUMMARY_LIST = "//li[contains(., 'Email is required')]"
 self.EMPTY_PASSWORD_ERROR_VALIDATION_SUMMARY_LIST = "//li[contains(., 'Please enter password')]"

 self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_VALIDATION_SUMMARY_LIST = "//a[@data-field='First and Last name separated by a space']"
 self.EMAIL_IS_NOT_VALID_VALIDATION_SUMMARY_LIST = "//a[@data-field='Email' and text()='Email is not valid']"
 self.PASSWORD_REQUIREMENTS_VALIDATION_SUMMARY_LIST = "//a[@data-field='Password' and contains(text(), '8 symbols')]"

 # --- Errors under inputs ---
 self.EMPTY_FIRST_AND_LAST_NAME_BOTTOM = "#FirstAndLastName-error"
 self.EMPTY_EMAIL_ERROR_BOTTOM = "#Email-error"
 self.EMPTY_PASSWORD_ERROR_BOTTOM = "//span[@id='Password-error' and contains(text(), 'Please enter password')]"

 self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_BOTTOM = "//span[@id='FirstAndLastName-error' and text()='First and Last name separated by a space']"
 self.EMAIL_IS_NOT_VALID_EMAIL_BOTTOM = "//span[@id='Email-error' and text()='Email is not valid']"
 self.PASSWORD_REQUIREMENTS_BOTTOM = "//span[@id='Password-error' and contains(text(), '8 symbols')]"

 # --- Form fields ---
 self.FIRST_AND_LAST_NAME_INPUT = "#FirstAndLastName"
 self.EMAIL_INPUT = "#Email"
 self.PASSWORD_INPUT = "#Password"
 self.REGISTER_SUBMIT_BUTTON = "//button[@type='submit']"

 def open_registration_url_pw(self):
 self.log_step("Navigating to registration page")

 # 1. Fast navigation: DOM ready is enough
 self.page.goto(
 "https://demos.telerik.com/kendo-ui/eshop/Account/Register",
 wait_until="domcontentloaded"
)

 # 2. Wait for a concrete form field instead of full page load
 self.page.wait_for_selector("input[name='Email']", state="visible", timeout=10000)

 return self

 def click_register_pw(self):
 # Use .first in case of duplicated submit buttons in the DOM.
 self.page.locator(self.REGISTER_SUBMIT_BUTTON).first.click()

 def trigger_required_errors_pw(self):
 # fill("") behaves like clear() + focus in Playwright.
 self.page.locator(self.FIRST_AND_LAST_NAME_INPUT).fill("")
 self.page.locator(self.EMAIL_INPUT).fill("")
 self.page.locator(self.PASSWORD_INPUT).fill("")
 self.click_register_pw()

 def trigger_format_errors_pw(self, incorrect_name, incorrect_email, short_password):
 self.page.locator(self.FIRST_AND_LAST_NAME_INPUT).fill(incorrect_name)
 self.page.locator(self.EMAIL_INPUT).fill(incorrect_email)
 self.page.locator(self.PASSWORD_INPUT).fill(short_password)
 self.click_register_pw()

 def is_visible_pw(self, selector):
 return self.page.locator(selector).first.is_visible()

 def get_text_pw(self, selector):
 return self.page.locator(selector).first.text_content()

 def expect_visible_pw(self, selector, timeout_ms=4000):
 expect(self.page.locator(selector).first).to_be_visible(timeout=timeout_ms)

 def expect_required_errors_pw(self):
 required_summary_selectors = [
 self.EMPTY_FIRST_AND_LAST_NAME_VALIDATION_SUMMARY_LIST,
 self.EMPTY_EMAIL_ERROR_VALIDATION_SUMMARY_LIST,
 self.EMPTY_PASSWORD_ERROR_VALIDATION_SUMMARY_LIST,
]
```

```

required_inline_selectors = [
 self.EMPTY_FIRST_AND_LAST_NAME_BOTTOM,
 self.EMPTY_EMAIL_ERROR_BOTTOM,
 self.EMPTY_PASSWORD_ERROR_BOTTOM,
]

for selector in required_summary_selectors + required_inline_selectors:
 self.expect_visible_pw(selector)

expect(self.page.locator(self.EMPTY_EMAIL_ERROR_BOTTOM).first).to_contain_text("Email is required")
expect(self.page.locator(self.EMPTY_PASSWORD_ERROR_BOTTOM).first).to_contain_text("Please enter a password")

def expect_format_errors_pw(self):
 format_summary_selectors = [
 self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_VALIDATION_SUMMARY_LIST,
 self.EMAIL_IS_NOT_VALID_VALIDATION_SUMMARY_LIST,
 self.PASSWORD_REQUIREMENTS_VALIDATION_SUMMARY_LIST,
]
 format_inline_selectors = [
 self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_BOTTOM,
 self.EMAIL_IS_NOT_VALID_EMAIL_BOTTOM,
 self.PASSWORD_REQUIREMENTS_BOTTOM,
]

 for selector in format_summary_selectors + format_inline_selectors:
 self.expect_visible_pw(selector)

expect(self.page.locator(self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_BOTTOM).first).to_contain_text("First and last name separated by a space")
expect(self.page.locator(self.EMAIL_IS_NOT_VALID_EMAIL_BOTTOM).first).to_contain_text("not valid email")
expect(self.page.locator(self.PASSWORD_REQUIREMENTS_BOTTOM).first).to_contain_text("8 symbols")

```

## File: pages\selenium\base\_page\_selenium.py

```
import allure
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
 ElementClickInterceptedException,
 ElementNotInteractableException,
 NoSuchElementException,
 StaleElementReferenceException,
 TimeoutException,
 WebDriverException,
)

class BasePage:

 def __init__(self, driver, timeout=3, poll_frequency=0.2):
 self.driver = driver
 self.default_timeout = timeout
 self.poll_frequency = poll_frequency
 self.wait = self._build_wait(timeout)

 def _build_wait(self, timeout):
 return WebDriverWait(
 self.driver,
 timeout,
 poll_frequency=self.poll_frequency,
 ignored_exceptions=(NoSuchElementException, StaleElementReferenceException),
)

 def _wait(self, condition, timeout=None):
 effective_timeout = timeout if timeout is not None else self.default_timeout
 return self._build_wait(effective_timeout).until(condition)

 def _log(self, level, message):
 print(f"[POM][{level}] {message}")

 def log_step(self, message):
 self._log("STEP", message)

 def log_info(self, message):
 self._log("INFO", message)

 def log_assert(self, message):
 self._log("ASSERT", message)

 def log_done(self, message):
 self._log("DONE", message)

 def log_warn(self, message):
 self._log("WARN", message)

 # Backward-compatible alias used in some page objects.
 def log_warning(self, message):
 self.log_warn(message)

 def log_error(self, message):
 self._log("ERROR", message)

 # ----- WAIT HELPERS -----
 def wait_for_visible(self, locator, timeout=None):
 return self._wait(EC.visibility_of_element_located(locator), timeout)

 def wait_for_clickable(self, locator, timeout=None):
 return self._wait(EC.element_to_be_clickable(locator), timeout)

 def wait_for_url(self, url, timeout=None):
 return self._wait(EC.url_contains(url), timeout)

 def wait_for_all_visible(self, locator, timeout=None):
 return self._wait(EC.visibility_of_all_elements_located(locator), timeout)

 def wait_for_presence(self, locator, timeout=None):
 return self._wait(EC.presence_of_element_located(locator), timeout)

 def wait_for_all_present(self, locator, timeout=None):
 return self._wait(EC.presence_of_all_elements_located(locator), timeout)

 def wait_for_page_load(self, timeout=4):
```



```

try:
 self._build_wait(timeout).until(
 lambda d: d.execute_script("return document.readyState") == "complete"
)
except TimeoutException:
 # Do not fail hard here; some apps remain usable while background scripts are still running.
 self._log("WARN", "Timed out waiting for document.readyState='complete'; continuing test.")

----- ACTIONS -----
def click(self, locator):
 self.wait_for_clickable(locator).click()

def safe_click(self, locator, retries=2):
 last_exception = None
 for attempt in range(retries):
 try:
 self.wait_for_clickable(locator).click()
 return
 except (
 TimeoutException,
 StaleElementReferenceException,
 ElementClickInterceptedException,
 ElementNotInteractableException,
) as exc:
 last_exception = exc
 if attempt < retries - 1:
 # Re-check clickability quickly without fixed sleeps.
 self.wait_for_clickable(locator, timeout=1.5)
 continue
 self._attach_screenshot("safe_click_error")
 raise TimeoutException(f"Failed to click element after {retries} attempts: {locator}") from last_exception

def type(self, locator, text):
 element = self.wait_for_visible(locator)
 element.clear()
 element.send_keys(text)

def get_text(self, locator):
 return self.wait_for_visible(locator).text

def open(self, url, retries=2):
 last_exception = None
 for attempt in range(retries):
 try:
 self.driver.get(url)
 self.wait_for_page_load(timeout=6)
 return
 except (TimeoutException, WebDriverException) as exc:
 last_exception = exc
 current_url = ""
 try:
 current_url = self.driver.current_url
 except Exception:
 pass
 # If renderer timed out but browser already navigated to target page, continue test.
 if current_url and url in current_url:
 self._log_warn(
 f"Navigation timeout occurred but URL is already reached: '{current_url}'. Continuing."
)
 return
 self._log_warn(
 f"Navigation attempt {attempt + 1}/{retries} failed for '{url}': {exc}"
)
 # Renderer timeout can leave Chrome in transient broken state; stop pending load and retry.
 try:
 self.driver.execute_script("window.stop();")
 except Exception:
 pass
 try:
 # Reset page context before next attempt to reduce repeated renderer stalls.
 self.driver.get("about:blank")
 except Exception:
 pass
 if attempt < retries - 1:
 continue
 raise last_exception

def scroll_to(self, locator, offset=-150):
 element = self.wait_for_visible(locator)
 self.driver.execute_script(
 "window.scrollTo(0, arguments[0].getBoundingClientRect().top + window.scrollY + arguments[1])"
)

```

```

 element,
 offset
)

def click_with_js(self, locator):
 element = self.wait_for_visible(locator)
 self.driver.execute_script("arguments[0].click();", element)

def get_title(self):
 return self.driver.title

----- FIND HELPERS -----
def find(self, locator):
 return self.driver.find_element(*locator)

def find_all(self, locator):
 return self.driver.find_elements(*locator)

----- ALLURE -----
def _attach_screenshot(self, name):
 allure.attach(
 self.driver.get_screenshot_as_png(),
 name=name,
 attachment_type=allure.attachment_type.PNG
)

```

## File: pages\selenium\header\_page\_selenium.py

```
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By
from selenium.common.exceptions import (
 ElementClickInterceptedException,
 NoSuchElementException,
 StaleElementReferenceException,
 TimeoutException,
)
from pages.selenium.base_page_selenium import BasePage

class HeaderPage(BasePage):
 # --- LOCATORS ---
 SHOPPING_CART_BTN = (By.XPATH, "//a[contains(@href, 'ShoppingCart')]")
 CATEGORIES_MENU = (By.XPATH, "//span[normalize-space()='Categories']")

 # Category locators
 ACCESSORIES_LINK = (By.XPATH, "//a[contains(@class, 'k-menu-link') and normalize-space()='Accessories']")
 BIKES_LINK = (By.XPATH, "//a[contains(@class, 'k-menu-link') and normalize-space()='Bikes']")
 CLOTHES_LINK = (By.XPATH, "//a[contains(@class, 'k-menu-link') and normalize-space()='Clothes']")
 COMPONENTS_LINK = (By.XPATH, "//a[contains(@class, 'k-menu-link') and normalize-space()='Components']")

 # --- BASIC METHODS ---

 def click_cart(self):
 self.click(self.SHOPPING_CART_BTN)

 def open_home_page(self):
 """Navigate user to the e-shop home page."""
 self.open("https://demos.telerik.com/kendo-ui/eshop")

 def get_url(self):
 return self.driver.current_url

 def assert_url_contains(self, expected_fragment):
 self.wait_for_url(expected_fragment, timeout=5)
 current_url = self.get_url()
 assert expected_fragment in current_url, (
 f"Expected URL containing '{expected_fragment}', got '{current_url}'"
)

 def validate_header_navigation(self):
 checks = [
 ("ShoppingCart", self.click_cart),
 ("eshop", self.open_home_page),
 ("Home/Accessories", self.open_accessories_category),
 ("Home/Bikes", self.open_bikes_category),
 ("Home/Clothing", self.open_clothes_category),
 ("Home/Components", self.open_components_category),
]

 for expected_fragment, action in checks:
 self.open_home_page()
 action()
 self.assert_url_contains(expected_fragment)

 # --- CATEGORY METHODS ---

 def open_accessories_category(self):
 self._hover_and_click(self.ACCESSORIES_LINK)

 def open_bikes_category(self):
 self._hover_and_click(self.BIKES_LINK)

 def open_clothes_category(self):
 self._hover_and_click(self.CLOTHES_LINK)

 def open_components_category(self):
 self._hover_and_click(self.COMPONENTS_LINK)

 # Shared hover-and-click logic
 def _hover_and_click(self, locator):
 categories = self.wait_for_visible(self.CATEGORIES_MENU, timeout=8)
 # Scroll to menu so it is visible and interactable.
```

```

self.driver.execute_script(
 "arguments[0].scrollIntoView({block: 'center', inline: 'nearest'});", categories
)

for attempt in range(3):
 actions = ActionChains(self.driver)
 actions.move_to_element(categories).pause(0.3).perform()

 try:
 self.wait_for_clickable(locator, timeout=4).click()
 return
 except (TimeoutException, StaleElementReferenceException, ElementClickInterceptedException):
 # Retry hover/click; menu rendering is flaky in headed mode.
 self.log_warn(f"Hover click attempt {attempt + 1}/3 failed for: {locator}")
 try:
 element = self.wait_for_visible(locator, timeout=2)
 self.driver.execute_script("arguments[0].click();", element)
 return
 except (TimeoutException, StaleElementReferenceException, ElementClickInterceptedException):
 continue

Last-resort fallback: read the target category URL and navigate directly.
self.log_warn(f"Falling back to direct navigation for category locator: {locator}")
self._open_category_link_direct(locator)

def _open_category_link_direct(self, locator):
 categories = self.wait_for_visible(self.CATEGORIES_MENU, timeout=8)
 ActionChains(self.driver).move_to_element(categories).pause(0.3).perform()
 self.wait_for_presence(locator, timeout=6)

 try:
 link = self.driver.find_element(*locator)
 href = link.get_attribute("href")
 except NoSuchElementException as exc:
 raise TimeoutException(f"Could not locate category link: {locator}") from exc

 if not href:
 raise TimeoutException(f"Category link has no href attribute: {locator}")

 self.open(href)

```

## File: pages\selenium\home\_page\_selenium.py

```
from selenium.webdriver.common.by import By
from pages.selenium.base_page_selenium import BasePage

class HomePage(BasePage):
 # --- URLs ---
 HOME_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop"
 REGISTRATION_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop/Account/Register"
 LOGIN_PAGE_URL = "https://demos.telerik.com/kendo-ui/eshop/Account/Login"
 CATEGORY_BIKES_URL = "https://demos.telerik.com/kendo-ui/eshop/Home/Bikes"

 # --- LOCATORS ---
 LOGOUT_BUTTON = (By.XPATH, "//a[contains(text(), 'Logout')]")
 MAIN_LOGIN_LINK = (By.XPATH, "//button[@type='submit' and contains(., 'Login')]")
 REGISTRATION_LOGIN_LINK = (By.XPATH, "//p[contains(text(), 'Already have an account?')]/following-sib")
 """Navigation methods."""
 def open_home_page(self):
 self.open(self.HOME_PAGE_URL)

 def open_register_page(self):
 self.open(self.REGISTRATION_PAGE_URL)

 def open_login_page(self):
 self.open(self.LOGIN_PAGE_URL)

 """Verification methods."""
 def is_logout_button_displayed(self):
 # Use BasePage wait helper to keep this check stable.
 # If logout does not appear, the wait raises TimeoutException.
 try:
 return self.wait_for_visible(self.LOGOUT_BUTTON).is_displayed()
 except:
 return False
```

## File: pages\selenium\login\_page\_selenium.py

```
from selenium.webdriver.common.by import By
from pages.selenium.base_page_selenium import BasePage
from selenium.webdriver.common.keys import Keys
from selenium.common.exceptions import TimeoutException

class LoginPage(BasePage):

 """Credentials"""
 ADMIN_TEST_USER_EMAIL = "jaxons.danniels@company.com"
 ADMIN_TEST_USER_PASS = "User1234"

 """Login page locators."""
 LOGIN_FIELD = (By.ID, "Email")
 PASSWORD_FIELD = (By.ID, "Password")
 LOGIN_BUTTON = (By.CSS_SELECTOR, ".k-form-submit")
 LOGIN_PAGE_URL_FRAGMENT = "Account/Login"
 LOGIN_ERROR_EMAIL = (By.XPATH, "//a[@data-field='Email' and contains(text(), 'not valid email')]")
 AUTHENTICATED_FAVORITES_LINK = (By.XPATH, "//a[contains(@href, '/Account/Favorites')]")
 """Methods using BasePage actions and waits."""

 def login_as_admin(self):
 for attempt in range(2):
 self.type(self.LOGIN_FIELD, self.ADMIN_TEST_USER_EMAIL)
 self.type(self.PASSWORD_FIELD, self.ADMIN_TEST_USER_PASS)

 if attempt == 0:
 # Primary path: click submit button.
 self.safe_click(self.LOGIN_BUTTON, retries=2)
 else:
 # Fallback path: submit with Enter for environments where click is flaky.
 self.wait_for_visible(self.PASSWORD_FIELD).send_keys(Keys.ENTER)

 try:
 self._wait(
 lambda d: self.LOGIN_PAGE_URL_FRAGMENT not in d.current_url,
 timeout=10,
)
 self._wait(
 lambda d: "Home Page" in d.title
 or len(d.find_elements(*self.AUTHENTICATED_FAVORITES_LINK)) > 0,
 timeout=10,
)
 return self
 except TimeoutException:
 if attempt == 0:
 # Retry once with Enter submit if click path did not complete authentication.
 self.open("https://demos.telerik.com/kendo-ui/eshop/Account/Login")
 continue
 raise

 return self

 def is_logged_in(self):
 try:
 self._wait(
 lambda d: self.LOGIN_PAGE_URL_FRAGMENT not in d.current_url
 and ("Home Page" in d.title or len(d.find_elements(*self.AUTHENTICATED_FAVORITES_LINK))
 > 0),
 timeout=10,
)
 return True
 except TimeoutException:
 return False
```

## File: pages\selenium\products\_page\_selenium.py

```
import contextlib
import re
import time

import allure
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
 ElementNotInteractableException,
 StaleElementReferenceException,
 TimeoutException,
 UnexpectedAlertPresentException,
)
from selenium.webdriver.support.wait import WebDriverWait

from pages.selenium.base_page_selenium import BasePage

class ProductsPage(BasePage):
 """Constants and links."""
 BIKE_MAIN_LINK = "https://demos.telerik.com/kendo-ui/eshop/Home/Bikes"
 SHOPPING_CART_URL = "https://demos.telerik.com/kendo-ui/eshop/Account/ShoppingCart"

 """Category locators."""
 BIKE_CATEGORY_TITLES = (By.XPATH, "//div[@class='category-heading']")
 MOUNTAINS_BIKES_CATEGORY = (By.XPATH, "//a[contains(@href, 'subCategory=Mountain Bikes')]")
 ROAD_BIKES_CATEGORY = (By.XPATH, "//a[contains(@href, 'subCategory=Road Bikes')]")
 TOURING_BIKES_CATEGORY = (By.XPATH, "//a[contains(@href, 'subCategory=Touring Bikes')]")

 """Filter locators."""
 # Radio buttons (state checks)
 DISCOUNTED_BIKES_RADIO_BUTTON = (By.XPATH, "//input[@type='radio' and @value='2']")
 ALL_BIKES_RADIO_BUTTON = (By.XPATH, "//input[@name='discountPicker' and @value='1']")

 """Filter labels (click targets)."""
 DISCOUNTED_BIKES_FILTER = (By.XPATH, "//label[contains(text(), 'Discounted items only')]")
 ALL_BIKES_FILTER = (By.XPATH, "//label[contains(text(), 'All')]")

 """Product card elements."""
 DISCOUNT_PERCENT_BADGE = (By.XPATH, "//span[@class='discount-pct']")
 PRODUCT_CARD = (By.XPATH, "//div[contains(@class, 'k-card')]")
 PAGER_INFO = (By.XPATH, "//span[@class='k-pager-info']")

 """Sorting locators."""
 SORT_DROPDOWN_TRIGGER = (By.XPATH, "//span[contains(@class, 'k-input-value-text')]")
 SORT_FILTER_LOW_TO_HIGH = (By.XPATH, "//span[@class='k-list-item-text' and text()='Price - Low to High']")
 SORT_FILTER_HIGH_TO_LOW = (By.XPATH, "//span[@class='k-list-item-text' and text()='Price - High to Low']")
 SORT_FILTER_A_TO_Z = (By.XPATH, "//span[@class='k-list-item-text' and text()='Name - A to Z']")
 SORT_FILTER_Z_TO_A = (By.XPATH, "//span[@class='k-list-item-text' and text()='Name - Z to A']")

 """Cart locators."""
 # "Add to Cart" on the first product card
 ADD_TO_CART_BUTTON = (By.XPATH, "(//button[contains(@class, 'add-to-cart') or contains(text(), 'Add to Cart')])")

 # Cart icon / navigation button
 CART_ICON = (By.XPATH, "//a[contains(@href, 'Cart')]")

 # Elements inside cart
 CART_ITEM_TITLES = (By.CSS_SELECTOR, ".cart-item-name, .k-card-title, .product-name")

 # Remove actions in cart (supports text/button variants).
 REMOVE_ITEM_BUTTONS = (By.CSS_SELECTOR, ".remove-product")
 # Optional empty-cart message (text differs slightly across UI variants).
 EMPTY_CART_MESSAGE = (
 By.XPATH,
 "//*[contains(translate(normalize-space(.), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), '')]"
)

 CART_LINE_ITEM_PRICES = (
 By.XPATH,
 "//td[contains(@class, 'final-price')] | //span[contains(@class, 'card-price')]"
)

 CART_TOTAL_PRICE = (By.ID, "subTotalValue")
 ALL_ADD_BUTTONS = (By.CLASS_NAME, "add-to-cart")
 """Cart actions."""
 def add_first_product_to_cart_and_verify(self):
```

```

 """Add first product to cart and verify it is present."""
 # 1. Capture first product name
 product_name = self.get_all_names()[0]
 self.log_info(f"Trying to add product: {product_name}")

 # 2. Click Add to Cart
 previous_count = self._current_cart_count()
 self.click(self.ADD_TO_CART_BUTTON)
 try:
 self._wait_for_cart_count_increment(previous_count, timeout=4)
 except TimeoutException:
 # Badge can lag on slow networks; cart assertion below still validates outcome.
 self.log_warn("Cart badge did not increment before opening cart; continuing.")

 # 3. Open cart
 self.click(self.CART_ICON)
 self.wait_for_url("Cart", timeout=6)
 self._wait(
 lambda d: len(d.find_elements(*self.CART_ITEM_TITLES)) > 0
 or len(d.find_elements(*self.EMPTY_CART_MESSAGE)) > 0,
 timeout=4,
)

 # 4. Verify product name inside cart
 CART_ITEM_TITLES = (By.CSS_SELECTOR, ".cart-item-name, .k-card-title, .product-name")

 # Use explicit visibility wait to avoid unstable empty-list reads.
 try:
 elements = self.wait_for_all_visible(CART_ITEM_TITLES, timeout=4)
 cart_names = [el.text.strip() for el in elements]
 except:
 cart_names = []

 self.log_info(f"Cart items found: {cart_names}")

 assert product_name in cart_names, (
 f"ERROR: Product '{product_name}' not found in cart. Found: {cart_names}"
)

 def _product_cards_with_add_to_cart(self):
 """Product cards that contain an add-to-cart control (order matches the grid)."""
 cards = self.driver.find_elements(*self.PRODUCT_CARD)
 with_cart = []
 for card in cards:
 try:
 if card.find_elements(By.CLASS_NAME, "add-to-cart"):
 with_cart.append(card)
 except Exception:
 continue
 return with_cart

 def add_multiple_products_and_verify_total(self, count=5):
 """Add N products and verify total cart value."""
 self.log_step(f"Starting cart total validation for {count} products")
 selected_prices = []

 # Add products
 for i in range(count):
 # Price and button from the same card – avoids mismatch between get_all_prices() and all .a
 self._wait_for_product_grid_ready(timeout=8)
 cards = self._product_cards_with_add_to_cart()
 if i >= len(cards):
 raise AssertionError(
 f"Could not find product card with add-to-cart for index {i}. "
 f"Only {len(cards)} cards found."
)
 card = cards[i]
 price_el = card.find_element(By.CSS_SELECTOR, ".card-price")
 clean_text = price_el.text.replace("$", "").replace(", ", "").strip()
 if not clean_text:
 raise AssertionError(f"Empty price on product card at index {i}.")
 price_value = float(clean_text)
 selected_prices.append(price_value)
 self.log_step(f"Adding product #{i + 1} priced at ${price_value}")
 add_loc = (
 By.XPATH,
 "(//div[contains(@class, 'k-card')][//button[contains(@class, 'add-to-cart')]])"
 f"[{i + 1}]/button[contains(@class, 'add-to-cart')]",
)
 previous_count = self._current_cart_count()
 self.scroll_to(add_loc)

```



```

self.safe_click(add_loc, retries=3)
try:
 self._wait_for_cart_count_increment(previous_count, timeout=12)
except TimeoutException:
 self.log_warn("Cart badge did not increment; retrying add (click + JS).")
 self.safe_click(add_loc, retries=2)
 try:
 self._wait_for_cart_count_increment(previous_count, timeout=10)
 except TimeoutException:
 self.click_with_js(add_loc)
 self._wait_for_cart_count_increment(previous_count, timeout=12)
expected_total = sum(selected_prices)
Header badge / tbody row count can disagree with Kendo layout; confirm server subtotal.
self._get_url_fast(self.SHOPPING_CART_URL, "ShoppingCart", part_timeout=15)
try:
 self._wait(
 lambda d: (
 (s := self._cart_subtotal_numeric()) is not None
 and abs(s - expected_total) < 0.08
),
 timeout=18,
)
except TimeoutException as exc:
 sub = self._cart_subtotal_numeric()
 lines = self._cart_shopping_grid_line_count()
 badge = self._current_cart_count()
 raise AssertionError(
 f"Cart subtotal did not match expected ${expected_total:.2f} "
 f"(got subtotal={sub}, grid_lines={lines}, header_badge={badge})."
) from exc

Already on ShoppingCart: skip extra navigation (same assertions as opening via icon).
if "ShoppingCart" not in (self.driver.current_url or ""):
 self.click(self.CART_ICON)

Wait for cart total to settle before reading value.
self.log_step("Waiting for cart total recalculation")
self._wait(
 lambda d: "$" in d.find_element(*self.CART_TOTAL_PRICE).text.strip(),
 timeout=6,
)

try:
 total_element = self.driver.find_element(*self.CART_TOTAL_PRICE)
 actual_total_text = total_element.text
 self.log_info(f"Raw cart total text: '{actual_total_text}'")

 # Convert "$1,234.56" or "$ 1,234.56" -> 1234.56
 normalized = re.sub(r"\s+", "", actual_total_text.replace("$", "").replace(",", ""))
 actual_total = float(normalized)
except Exception as e:
 # Preserve context if total cannot be parsed.
 self.log_error(f"Failed to read cart total: {e}")
 raise

cart_line_prices = self.get_cart_line_item_prices()
cart_line_total = sum(cart_line_prices)

self.log_assert(f"Expected total: ${expected_total:.2f}")
self.log_assert(f"Actual total: ${actual_total:.2f}")
self.log_assert(f"Cart line-items sum: ${cart_line_total:.2f}")

assert round(cart_line_total, 2) == round(actual_total, 2), (
 f"CART LINE SUM ERROR: line-items total {cart_line_total} does not match subtotal {actual_t
)

Use round() to avoid float precision artifacts.
assert round(actual_total, 2) == round(expected_total, 2), \
 f"TOTAL ERROR: got {actual_total}, expected {expected_total}"
self.log_done("Cart total is correct")

def _current_cart_count(self):
 """Return current numeric cart header badge; avoid list-page discount .k-badge (e.g. 20% -> 20).
 for locator in (
 (By.ID, "shopping-cart-badge"),
 (By.CSS_SELECTOR, "a[href*='Cart'] .k-badge, a[href*='ShoppingCart'] .k-badge, .cart-count"
):
 for badge in self.driver.find_elements(*locator):
 try:
 text = badge.text.strip()
 if text.isdigit():

```

```

 return int(text)
 except Exception:
 continue
 return 0

def _wait_for_cart_count_increment(self, previous_count, timeout=2):
 self._wait(lambda d: self._current_cart_count() > previous_count, timeout=timeout)

def _wait_for_header_cart_badge(self, expected_count, timeout=12):
 """Wait until the header shopping-cart badge shows exactly expected_count (slow on Windows/demo)
 self._wait(lambda d: self._current_cart_count() == expected_count, timeout=timeout)

def _get_remove_buttons(self):
 """Return visible remove controls from supported cart templates."""
 fallback_locators = [
 self.REMOVE_ITEM_BUTTONS,
 (By.XPATH, "//p[normalize-space()='Remove']"),
 (By.XPATH, "//button[contains(@class, 'remove-product') or normalize-space()='Remove']"),
 # Telerik eShop: shopping_cart.js uses id="remove_<itemId>" on a control in the Kendo grid
 (By.CSS_SELECTOR, "[id^='remove_']"),
 (By.XPATH, "//*[id[starts-with(., 'remove_')]]"),
]
 for locator in fallback_locators:
 elements = self.driver.find_elements(*locator)
 visible_elements = []
 for element in elements:
 try:
 if element.is_displayed():
 visible_elements.append(element)
 except Exception:
 # Element can become stale during cart re-render; ignore and keep searching.
 continue
 if visible_elements:
 return visible_elements
 return []

def _is_cart_empty_message_visible(self):
 """DOM message such as 'Your shopping cart is empty'."""
 self._accept_any_alert()
 if self.driver.find_elements(*self.EMPTY_CART_MESSAGE):
 return True
 # Alternate copy on some builds / locales
 alt = (
 By.XPATH,
 "//*[contains(translate(normalize-space(.), "
 "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), "
 "'nothing in your cart') or contains(translate(normalize-space(.), "
 "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'shopping cart is empty')]]",
)
 return len(self.driver.find_elements(*alt)) > 0

def _cart_kendo_item_count(self):
 """Legacy Kendo dataSource length; can lie vs DOM – prefer _cart_shopping_grid_line_count."""
 self._accept_any_alert()
 with contextlib.suppress(Exception):
 n = self.driver.execute_script(
 """
 if (!window.jQuery) { return -1; }
 var g = jQuery("#shoppingCartGrid").data("kendoGrid");
 if (!g) { return -1; }
 try {
 var ds = g.dataSource;
 if (ds && ds.data && typeof ds.data === "function") { return ds.data().length; }
 if (ds && typeof ds.total === "function") { return ds.total(); }
 return g.items().length;
 } catch (e) { return -1; }
 """
)
 if n is not None:
 return int(n)
 return -1

def _cart_shopping_grid_line_count(self):
 """
 Count tbody data rows on #shoppingCartGrid (excludes k-grid-norecords template rows).
 More reliable than dataSource.total() when the grid UI and server state disagree.
 """
 self._accept_any_alert()
 with contextlib.suppress(Exception):
 n = self.driver.execute_script(
 r"""

```

```

 var grid = document.querySelector("#shoppingCartGrid");
 if (!grid) { return -1; }
 var tbody = grid.querySelector("tbody");
 if (!tbody) { return -1; }
 var rows = tbody.querySelectorAll("tr");
 var c = 0;
 for (var i = 0; i < rows.length; i++) {
 var tr = rows[i];
 if (tr.classList.contains("k-grid-norecords")) { continue; }
 var tds = tr.querySelectorAll("td");
 /* Merged Kendo rows may use a single wide td – count any data row with cells. */
 if (tds.length >= 1) { c++; }
 }
 return c;
 """
)
 if n is not None:
 return int(n)
 return -1

def _cart_subtotal_numeric(self):
 """Parse #subTotalValue; None if missing or not a money amount."""
 self._accept_any_alert()
 with contextlib.suppress(Exception):
 el = self.driver.find_element(*self.CART_TOTAL_PRICE)
 txt = el.text.strip()
 if not txt or not re.search(r"\d", txt):
 return None
 normalized = re.sub(r"\s+", "", txt.replace("$", "").replace(",", ""))
 return float(normalized)
 return None

def _is_cart_effectively_empty(self):
 """
 True only when the shopping cart page shows no billable lines:
 no grid line rows, no remove controls, subtotal ~0, header badge 0.
 """
 self._accept_any_alert()
 if self._get_remove_buttons():
 return False

 line_rows = self._cart_shopping_grid_line_count()
 if line_rows > 0:
 return False
 if line_rows < 0:
 # Grid not in DOM yet – not confidently empty.
 if self.driver.find_elements(By.ID, "shoppingCartGrid"):
 return False

 sub = self._cart_subtotal_numeric()
 if sub is not None and sub > 0.02:
 return False

 k = self._cart_kendo_item_count()
 if k is not None and k > 0:
 return False

 if self._current_cart_count() > 0:
 return False

 # Do not treat "0 counted rows + grid in DOM" as empty: row counters can miss merged cells;
 # require either the empty copy or a confirmed zero subtotal.
 if self._is_cart_empty_message_visible():
 return True

 sub_final = self._cart_subtotal_numeric()
 if (
 line_rows == 0
 and self.driver.find_elements(By.ID, "shoppingCartGrid")
 and sub_final is not None
 and sub_final <= 0.02
):
 return True

 return False

def _verify_shopping_cart_empty_via_reload(self, timeout=12):
 """
 Load ShoppingCart twice from the server. Catches stale DOM / cache where the first
 paint looks empty but line items still exist (seen on Windows + slow Telerik demo).
 """

```

```

 for pass_num in range(2):
 self._get_url_fast(
 self.SHOPPING_CART_URL,
 "ShoppingCart",
 part_timeout=max(12, timeout),
)
 self._wait(lambda d: self._is_cart_effectively_empty(), timeout=timeout)
 self.log_info(f"Shopping cart empty check pass {pass_num + 1}/2 OK.")

def assert_shopping_cart_empty(self, timeout=12):
 """
 Open the cart page and fail fast if line items or totals still indicate a non-empty cart.
 Call after clear_cart() when the test depends on a zero baseline.
 On success, navigates to Home/Bikes (same as clear_cart()) so the next step can open categories.
 """
 try:
 self._verify_shopping_cart_empty_via_reload(timeout=timeout)
 except TimeoutException as exc:
 n = self._cart_kendo_item_count()
 lines = self._cart_shopping_grid_line_count()
 sub = self._cart_subtotal_numeric()
 badge = self._current_cart_count()
 raise AssertionError(
 f"Shopping cart is not empty (grid_lines={lines}, kendo={n}, "
 f"subtotal={sub}, header_badge={badge})."
) from exc
 # Match clear_cart() post-condition: listing tests expect Home/Bikes context.
 self._open_bikes_landing_from_clear()
 self._wait_for_header_cart_badge(0, timeout=15)

def _clear_cart_via_remove_buttons(self, max_clicks=40):
 """Remove line items via visible Remove controls (real UI / server sync)."""
 for _ in range(max_clicks):
 if self._is_cart_effectively_empty():
 return True
 buttons = self._get_remove_buttons()
 if not buttons:
 break
 btn = buttons[0]
 try:
 self.driver.execute_script(
 "arguments[0].scrollIntoView({block: 'center', inline: 'nearest'});", btn
)
 except Exception:
 pass
 try:
 btn.click()
 except (StaleElementReferenceException, ElementNotInteractableException):
 with contextlib.suppress(Exception):
 self.driver.execute_script("arguments[0].click();", btn)
 self._accept_any_alert()
 try:
 self._wait(
 lambda d: self._is_cart_effectively_empty()
 or len(self._get_remove_buttons()) < len(buttons),
 timeout=5,
)
 except (TimeoutException, UnexpectedAlertPresentException):
 self._accept_any_alert()
 self.log_warn("Remove click did not update cart DOM in time; retrying.")
 return self._is_cart_effectively_empty()

def _get_page_load_timeout(self):
 t = 4.0
 with contextlib.suppress(Exception):
 timeouts = getattr(self.driver, "timeouts", None)
 if timeouts is not None and getattr(timeouts, "page_load", None) is not None:
 t = float(timeouts.page_load)
 return t

def _set_page_load_timeout(self, seconds):
 with contextlib.suppress(Exception):
 self.driver.set_page_load_timeout(seconds)

def _accept_any_alert(self):
 with contextlib.suppress(Exception):
 self.driver.switch_to.alert.accept()

def _get_url_fast(self, url, url_part, part_timeout=8):
 """One navigation with a bounded page-load timeout; avoids long BasePage.open() retries."""
 self._accept_any_alert()

```

```

old = self._get_page_load_timeout()
with contextlib.suppress(Exception):
 self._set_page_load_timeout(max(10, part_timeout + 2))
try:
 with contextlib.suppress(Exception):
 self.driver.get(url)
except Exception:
 # Brief explicit wait for navigation to settle instead of a blind sleep.
 with contextlib.suppress(TimeoutException):
 self.wait_for_url(url_part, timeout=2)
with contextlib.suppress(TimeoutException):
 self.wait_for_url(url_part, timeout=part_timeout)
with contextlib.suppress(Exception):
 self._set_page_load_timeout(old)

def _open_bikes_landing_from_clear(self):
 self._get_url_fast(self.BIKE_MAIN_LINK, "Home/Bikes", part_timeout=10)
 self._wait_for_bikes_landing_ready(timeout=8)
 # Header badge can lag behind server-side cart clear on slow networks / Windows.
 try:
 self._wait_for_header_cart_badge(0, timeout=15)
 except TimeoutException:
 self.log_warn("Header cart badge did not show 0 after returning to Bikes; continuing.")

def _clear_shopping_cart_grid_kendo(self):
 """
 Pop all Kendo grid rows and sync to the server (Telerik eShop /js/shopping_cart.js).
 """
 with contextlib.suppress(Exception):
 return int(
 self.driver.execute_script(
 """
 if (!window.jQuery) { return 0; }
 var oldc = window.confirm, n = 0;
 try {
 window.confirm = function() { return true; };
 var g = jQuery("#shoppingCartGrid").data("kendoGrid");
 if (!g) { return 0; }
 var guard = 300;
 while (guard-- > 0) {
 var rows = g.tbody ? g.tbody.find("tr") : jQuery();
 if (!rows || !rows.length) { break; }
 g.removeRow(jQuery(rows[0]));
 n++;
 g.dataSource.sync();
 }
 } finally {
 window.confirm = oldc;
 }
 if (typeof calculateShoppingCartTotal === "function") { try { calculateShoppingCartTotal(); }
 if (typeof getShoppingCartItemsCount === "function") { try { getShoppingCartItemsCount(); }
 return n;
 """
)
 or 0
)
 return 0

def clear_cart(self):
 """
 Empty the in-app shopping cart (server state), then return to the bikes page.

 Prefer the same path as a user: visible Remove controls (locators), then optional
 Kendo bulk clear if rows remain. Raises if the cart cannot be emptied.
 """
 self.log_step("Clearing shopping cart...")

 with contextlib.suppress(Exception):
 self.driver.execute_script("localStorage.clear(); sessionStorage.clear();")

 for attempt in range(4):
 self._get_url_fast(
 self.SHOPPING_CART_URL,
 "ShoppingCart",
 part_timeout=12,
)
 try:
 self._wait(
 lambda d: self._is_cart_effectively_empty()
 or len(d.find_elements(By.ID, "shoppingCartGrid")) > 0
 or len(d.find_elements(*self.REMOVE_ITEM_BUTTONS)) > 0
)

```

```

 or len(d.find_elements(By.CSS_SELECTOR, "[id^='remove_']")) > 0,
 timeout=10,
)
except TimeoutException:
 self.log_warn("Cart page shell did not stabilize in time; attempting clear anyway.")

if self._is_cart_effectively_empty():
 self._open_bikes_landing_from_clear()
 return

1) Locator-driven removes first (stable, mirrors manual testing).
self._clear_cart_via_remove_buttons()
try:
 self._wait(lambda d: self._is_cart_effectively_empty(), timeout=12)
except TimeoutException:
 self.log_warn(
 f"Cart still has items after Remove-locator pass (attempt {attempt + 1}/4)."
)

if self._is_cart_effectively_empty():
 self._open_bikes_landing_from_clear()
 return

2) Bulk Kendo clear, then Remove again for anything left.
self._clear_shopping_cart_grid_kendo()
self._clear_cart_via_remove_buttons()
try:
 self._wait(lambda d: self._is_cart_effectively_empty(), timeout=12)
except TimeoutException:
 self.log_warn(
 f"Cart still not empty after Kendo + Remove pass (attempt {attempt + 1}/4)."
)

if self._is_cart_effectively_empty():
 self._open_bikes_landing_from_clear()
 return

self._get_url_fast(
 self.SHOPPING_CART_URL,
 "ShoppingCart",
 part_timeout=12,
)
if not self._is_cart_effectively_empty():
 raise AssertionError(
 "clear_cart() could not empty the shopping cart: "
 "Remove locators and Kendo grid clear were insufficient. "
 "See assert_shopping_cart_empty() for a hard check in tests."
)
self._open_bikes_landing_from_clear()

"""Navigation methods."""

def open_bikes_main_link(self):
 """Open bikes landing page."""
 self.open(self.BIKE_MAIN_LINK)
 self._wait_for_bikes_landing_ready(timeout=5)

def open_mountain_bikes(self):
 """Open Mountain Bikes category."""
 self.click(self.MOUNTAINS_BIKES_CATEGORY)
 self._wait_for_product_grid_ready(timeout=5)

def _wait_for_bikes_landing_ready(self, timeout=5):
 """Wait for bikes landing page regardless of whether pager is visible yet."""
 self.wait_for_url("Home/Bikes", timeout=timeout)
 self._wait(
 lambda d: len(d.find_elements(*self.BIKE_CATEGORY_TITLES)) > 0
 or len(d.find_elements(*self.PRODUCT_CARD)) > 0
 or len(d.find_elements(*self.PAGER_INFO)) > 0,
 timeout=timeout,
)

def _wait_for_product_grid_ready(self, timeout=5):
 """Wait for product cards and related listing markers to appear."""
 # `len(...) >= 0` was always true, so the wait effectively required only cards
 # and could return before pager/Kendo finished rendering (flaky clicks/asserts).
 self._wait(
 lambda d: len(d.find_elements(*self.PRODUCT_CARD)) > 0
 and (

```

```

 len(d.find_elements(*self.PAGER_INFO)) > 0
 or len(d.find_elements(*self.DISCOUNT_PERCENT_BADGE)) > 0
),
 timeout=timeout,
)

"""Data getters."""

def get_all_prices(self):
 """Collect all product prices and convert to float."""
 PRICE_LOCATOR = (By.XPATH, "//div[@class='card-price']")
 elements = self.driver.find_elements(*PRICE_LOCATOR)
 prices = []
 for el in elements:
 clean_text = el.text.replace('$', '').replace(',', '').strip()
 if clean_text:
 prices.append(float(clean_text))
 return prices

def get_all_names(self):
 """Collect all product names."""
 NAME_LOCATOR = (By.XPATH, "//div[@class='k-card-title']")
 elements = self.driver.find_elements(*NAME_LOCATOR)
 return [el.text.strip() for el in elements if el.text]

def get_all_discount_labels(self):
 """Collect discount badge labels (e.g., '20% off')."""
 elements = self.driver.find_elements(*self.DISCOUNT_PERCENT_BADGE)
 return [el.text.strip() for el in elements if el.text]

@staticmethod
def _parse_price_text(text):
 match = re.search(r"([0-9]+(?:\.[0-9]{3})?(?:\.[0-9]{2})?)", text.replace("$", ""))
 if not match:
 raise ValueError(f"Could not parse price from text: '{text}'")
 return float(match.group(1).replace(".", ""))

def get_cart_line_item_prices(self):
 self.log_info("[POM] Reading product line prices from the cart...")

 # Use wait_for_all_visible because find_elements does not wait and often returns [] for async UI
 try:
 # CART_LINE_ITEM_PRICES is the shared locator for cart line prices
 price_elements = self.wait_for_all_visible(self.CART_LINE_ITEM_PRICES, timeout=7)
 except Exception as e:
 self.log_warn(f"[POM] Timed out waiting for cart line prices: {e}. Falling back to find_elements")
 price_elements = self.driver.find_elements(*self.CART_LINE_ITEM_PRICES)

 prices = []
 for element in price_elements:
 try:
 # Use visible elements only
 if element.is_displayed():
 text = element.text.strip()
 if text:
 prices.append(self._parse_price_text(text))
 except Exception as e:
 self.log_warn(f"[POM] Error parsing a single line price: {e}")
 continue

 if not prices:
 # Screenshot in Allure helps when debugging
 allure.attach(
 self.driver.get_screenshot_as_png(),
 name="failed_cart_prices_view",
 attachment_type=allure.attachment_type.PNG
)
 # Attach HTML so reports show what the DOM actually contained
 allure.attach(self.driver.page_source, name="page_source", attachment_type=allure.attachment_type.HTML)
 raise AssertionError("No product prices found in the cart.")

 self.log_info(f"[POM] Read {len(prices)} prices: {prices}")
 return prices

def get_total_count_from_pager(self):
 """Extract total item count from pager text."""
 text_info = self.driver.find_element(*self.PAGER_INFO).text
 # Split example: ['1', '-', '12', 'of', '32', 'items'] -> index 4
 parts = text_info.split()
 return int(parts[4])

def get_bike_category_titles(self):

```

```

 """Collect text for bike category headers."""
 elements = self.wait_for_all_visible(self.BIKE_CATEGORY_TITLES)
 return [el.text.strip() for el in elements]

def assert_expected_bike_categories(self, expected_titles=None):
 titles = self.get_bike_category_titles()
 self.log_info(f"Bike categories found: {titles}")
 expected = expected_titles or ["Mountain Bikes", "Road Bikes", "Touring Bikes"]
 for expected_title in expected:
 assert expected_title in titles, f"Expected category '{expected_title}' in {titles}"

"""Counter helpers."""

def count_visible_bikes(self):
 """Count cards with non-empty visible title text."""
 # Read title elements inside cards.
 titles = self.driver.find_elements(By.XPATH, "//div[contains(@class, 'k-card')]/div[@class='k-"]

 # Count only entries with non-empty text.
 visible_titles = [t for t in titles if t.text.strip() != ""]

 return len(visible_titles)

def count_badges(self):
 """Count discount percentage badges visible on screen."""
 return len(self.get_all_discount_labels())

"""Validation helpers."""

def verify_discount_filter(self):
 """Validate discounted filter consistency."""
 self.log_step("Applying discounted-items filter")
 self.click(self.DISCOUNTED_BIKES_FILTER)
 self._wait_for_product_grid_ready(timeout=6)

 bikes_count = self.count_visible_bikes()
 badges_count = self.count_badges()

 self.log_info(f"Discount filter debug: products={bikes_count}, badges={badges_count}")

 assert bikes_count > 0, "ERROR: Discount filter returned no results."
 assert bikes_count == badges_count, (
 f"FILTER ERROR: displayed {bikes_count} products, "
 f"but only {badges_count} has a discount badge."
)
 self.log_done("All visible products have discount badges")

def all_sorting_options(self):
 """Validate all sorting options one-by-one."""
 scenarios = [
 (self.SORT_FILTER_A_TO_Z, "name_asc"),
 (self.SORT_FILTER_Z_TO_A, "name_desc"),
 (self.SORT_FILTER_LOW_TO_HIGH, "price_asc"),
 (self.SORT_FILTER_HIGH_TO_LOW, "price_desc")
]

 for locator, sort_type in scenarios:
 self.click(self.SORT_DROPDOWN_TRIGGER)
 try:
 option_element = self.wait_for_visible(locator, timeout=4)
 except TimeoutException:
 # Kendo dropdown can collapse right after opening; reopen and use presence fallback.
 self.click_with_js(self.SORT_DROPDOWN_TRIGGER)
 option_element = self.wait_for_presence(locator, timeout=4)

 option_name = option_element.text
 try:
 self.safe_click(locator, retries=3)
 except TimeoutException:
 self.log_warn(f"Standard sort-option click failed; using JS click for '{option_name}'")
 try:
 fresh = self.wait_for_visible(locator, timeout=3)
 self.driver.execute_script("arguments[0].click();", fresh)
 except (TimeoutException, StaleElementReferenceException):
 self.driver.execute_script("arguments[0].click();", option_element)

 self._wait(
 lambda d: option_name.lower() in d.find_element(*self.SORT_DROPDOWN_TRIGGER).text.lower()
 timeout=5,
)
 if "Price" in option_name:

```



```

 actual = self.get_all_prices()
 is_desc = (sort_type == "price_desc")
 assert actual == sorted(actual, reverse=is_desc), f"Price sorting error: {option_name}"
 elif "Name" in option_name:
 actual = self.get_all_names()
 is_desc = (sort_type == "name_desc")
 assert actual == sorted(actual, reverse=is_desc), f"Name sorting error: {option_name}"

 self.log_done(f"Sorting '{option_name}' works correctly")

def assert_products_available_for_all_filter(self):
 self.click(self.ALL_BIKES_FILTER)
 self._wait_for_product_grid_ready(timeout=5)
 all_items_count = self.get_total_count_from_pager()
 assert all_items_count > 0, f"Expected all-items count > 0, got {all_items_count}"
 return all_items_count

def assert_discounted_filter_consistency(self, expected_all_count=None):
 self.click(self.DISCOUNTED_BIKES_FILTER)
 self._wait_for_product_grid_ready(timeout=5)
 discounted_pager_count = self.get_total_count_from_pager()
 discounted_badges_count = self.count_badges()
 discounted_cards_count = self.count_visible_bikes()
 if expected_all_count is not None:
 assert discounted_pager_count <= expected_all_count, (
 f"Expected discounted count ({discounted_pager_count}) <= all items ({expected_all_count}"
)
 assert discounted_pager_count > 0, (
 f"Expected discounted pager count > 0, got {discounted_pager_count}"
)
 assert discounted_badges_count == discounted_pager_count, (
 f"Expected discounted badges ({discounted_badges_count}) to match pager ({discounted_pager_count}"
)
 assert discounted_cards_count == discounted_pager_count, (
 f"Expected discounted cards ({discounted_cards_count}) to match pager ({discounted_pager_count}"
)

def assert_filter_counts_consistent(self):
 all_items_count = self.assert_products_available_for_all_filter()
 self.assert_discounted_filter_consistency(expected_all_count=all_items_count)

```

## File: pages\selenium\registration\_page\_selenium.py

```
from selenium.webdriver.common.by import By
from pages.selenium.base_page_selenium import BasePage

class RegistrationPage(BasePage):
 """
 Web element locators - Registration - Errors Registration
 """

 # Validation summary errors

 # State: empty fields
 EMPTY_FIRST_AND_LAST_NAME_VALIDATION_SUMMARY_LIST = (By.XPATH, "//li[contains(., 'First and Last name is required')]")
 EMPTY_EMAIL_ERROR_VALIDATION_SUMMARY_LIST = (By.XPATH, "//li[contains(., 'Email is required')]")

 EMPTY_PASSWORD_ERROR_VALIDATION_SUMMARY_LIST = (By.XPATH, "//li[contains(., 'Please enter password')]")

 # State: wrong formats
 FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_VALIDATION_SUMMARY_LIST = (By.XPATH,
 "//a[@data-field='FirstAndLastName' and text()='First and last name should be separated by a space']")
 EMAIL_IS_NOT_VALID_VALIDATION_SUMMARY_LIST = (By.XPATH,
 "//a[@data-field='Email' and text()='Email is not valid']")
 PASSWORD_REQUIREMENTS_VALIDATION_SUMMARY_LIST = (By.XPATH,
 "//a[@data-field='Password' and contains(text(), '8 symbols')]")

 """
 Web element locators - Registration - errors under inputs
 """
 # --- Errors under inputs ---
 # State: empty fields errors
 # 1
 EMPTY_FIRST_AND_LAST_NAME_BOTTOM = (By.ID, "FirstAndLastName-error")
 # 2
 EMPTY_EMAIL_ERROR_BOTTOM = (By.ID, "Email-error")
 # 3
 EMPTY_PASSWORD_ERROR_BOTTOM = (By.XPATH,
 "//span[@id='Password-error' and contains(text(), 'Please enter password')]")

 # State: wrong format errors
 # 4
 FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_BOTTOM = (By.XPATH,
 "//span[@id='FirstAndLastName-error' and contains(text(), 'First and last name should be separated by a space')]")
 # 5
 EMAIL_IS_NOT_VALID_EMAIL_BOTTOM = (By.XPATH, "//span[@id='Email-error' and text()='Email is not valid']")
 # 6
 PASSWORD_REQUIREMENTS_BOTTOM = (By.XPATH, "//span[@id='Password-error' and contains(text(), '8 symbols')]")

 """
 Web element locators - FORM Register and Login
 """

 FIRST_AND_LAST_NAME_INPUT = (By.ID, "FirstAndLastName")
 EMAIL_INPUT = (By.ID, "Email")
 PASSWORD_INPUT = (By.ID, "Password")

 """
 Web element locators - buttons
 """

 LOGIN_BUTTON_REGISTRATION = (By.XPATH, "//a[@href='/kendo-ui/eshop/Account/Login']")
 REGISTER_SUBMIT_BUTTON = (By.XPATH, "//button[@type='submit']")

 """
 Methods
 """

 def click_register(self):
 self.wait_for_clickable(self.REGISTER_SUBMIT_BUTTON).click()

 def trigger_required_errors(self):
 self.wait_for_visible(self.FIRST_AND_LAST_NAME_INPUT).clear()
 self.wait_for_visible(self.EMAIL_INPUT).clear()
 self.wait_for_visible(self.PASSWORD_INPUT).clear()
 self.click_register()

 def trigger_format_errors(self, incorrect_name, incorrect_email, short_password):
 self.wait_for_visible(self.FIRST_AND_LAST_NAME_INPUT).send_keys(incorrect_name)
```

```

self.wait_for_visible(self.EMAIL_INPUT).send_keys(incorrect_email)
self.wait_for_visible(self.PASSWORD_INPUT).send_keys(short_password)
self.click_register()

def open_registration_url(self):
 self.open("https://demos.telerik.com/kendo-ui/eshop/Account/Register")

def get_required_error_elements(self):
 return {
 "summary": {
 "name": self.wait_for_visible(self.EMPTY_FIRST_AND_LAST_NAME_VALIDATION_SUMMARY_LIST),
 "email": self.wait_for_visible(self.EMPTY_EMAIL_ERROR_VALIDATION_SUMMARY_LIST),
 "password": self.wait_for_visible(self.EMPTY_PASSWORD_ERROR_VALIDATION_SUMMARY_LIST),
 },
 "inline": {
 "name": self.wait_for_visible(self.EMPTY_FIRST_AND_LAST_NAME_BOTTOM),
 "email": self.wait_for_visible(self.EMPTY_EMAIL_ERROR_BOTTOM),
 "password": self.wait_for_visible(self.EMPTY_PASSWORD_ERROR_BOTTOM),
 },
 }

def assert_required_errors(self):
 required_errors = self.get_required_error_elements()
 for element in required_errors["summary"].values():
 assert element.is_displayed()
 assert required_errors["inline"]["name"].is_displayed()
 assert "Email is required" in required_errors["inline"]["email"].text
 assert "Please enter password" in required_errors["inline"]["password"].text

def get_format_error_elements(self):
 return {
 "summary": {
 "name": self.wait_for_visible(self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_VALIDATION_SUMMARY_LIST),
 "email": self.wait_for_visible(self.EMAIL_IS_NOT_VALID_VALIDATION_SUMMARY_LIST),
 "password": self.wait_for_visible(self.PASSWORD_REQUIREMENTS_VALIDATION_SUMMARY_LIST),
 },
 "inline": {
 "name": self.wait_for_visible(self.FIRST_AND_LAST_NAME_SEPARATED_BY_A_SPACE_BOTTOM),
 "email": self.wait_for_visible(self.EMAIL_IS_NOT_VALID_EMAIL_BOTTOM),
 "password": self.wait_for_visible(self.PASSWORD_REQUIREMENTS_BOTTOM),
 },
 }

def assert_format_errors(self):
 format_errors = self.get_format_error_elements()
 for element in format_errors["summary"].values():
 assert element.is_displayed()
 assert "separated by a space" in format_errors["inline"]["name"].text
 assert "not valid email" in format_errors["inline"]["email"].text
 assert "8 symbols" in format_errors["inline"]["password"].text

```

**File: pages\selenium\\_\_init\_\_.py**

## File: playwright\_tests\conftest.py

```
import pytest
import allure
from pages.playwright.header_page_playwright import HeaderPagePw
from pages.playwright.home_page_playwright import HomePagePw
from pages.playwright.login_page_playwright import LoginPagePw
from pages.playwright.products_page_playwright import ProductPagePw
from pages.playwright.registration_page_playwright import RegistrationPagePw

@pytest.fixture
def home_page_pw(page):
 return HomePagePw(page)

@pytest.fixture
def login_page_pw(page):
 return LoginPagePw(page)

@pytest.fixture
def header_page_pw(page):
 return HeaderPagePw(page)

@pytest.fixture
def product_page_pw(page):
 return ProductPagePw(page)

@pytest.fixture
def registration_page_pw(page):
 return RegistrationPagePw(page)

@pytest.fixture(scope="session")
def browser_context_args(browser_context_args):
 return {
 **browser_context_args,
 "viewport": {"width": 1920, "height": 1080},
 "base_url": "https://demos.telerik.com/kendo-ui/eshop"
 }

@pytest.fixture(scope="session")
def browser_type_launch_args():
 return {
 "headless": False,
 "slow_mo": 5,
 "args": ["--window-size", "--window-size=1920,1080"]
 }

@pytest.fixture(scope="session")
def browser_context_args():
 return {
 # viewport=None lets the browser control window size
 "viewport": None,
 "base_url": "https://demos.telerik.com/kendo-ui/eshop"
 }

@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
 outcome = yield
 rep = outcome.get_result()
 _ = call

 if rep.when == "call" and rep.failed:
 # Get the test's `page` fixture
 page = item.funcargs.get("page")
 if page:
 try:
 # full_page=False captures the viewport only
 # timeout=5000 avoids hanging forever
 screenshot = page.screenshot(full_page=False, timeout=5000)
 allure.attach(
 screenshot,
```

```
 name="failure_screenshot_pw",
 attachment_type=allure.attachment_type.PNG
)
except Exception as e:
 print(f"\n[Allure] Failed to capture failure screenshot: {e}")
```

## File: playwright\_tests\test\_adding\_multiple\_items\_to\_basket\_playwright.py

```
import allure
import pytest

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
@pytest.mark.flaky(reruns=2, reruns_delay=2)
def test_multiple_products_total_calculation_pw(home_page_pw, login_page_pw, product_page_pw):
 print("[STEP] Open login page and authenticate.")
 # Use _pw-suffixed page object methods
 home_page_pw.open_login_page_pw()
 login_page_pw.login_as_admin_pw()

 print("[STEP] Navigate to Bikes and clear cart.")
 product_page_pw.page.goto(product_page_pw.BIKE_MAIN_LINK)
 product_page_pw.clear_cart_pw()

 print("[STEP] Open mountain bikes category.")
 product_page_pw.open_mountain_bikes_pw()

 print("[STEP] Add multiple products and validate computed cart total.")
 product_page_pw.add_multiple_products_and_verify_total_pw(count=5)
```

## File: playwright\_tests\test\_basket\_playwirght.py

```
import allure

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
def test_add_to_cart_flow_pw(home_page_pw, login_page_pw, product_page_pw):
 print("\n[PW][STEP] Open login page and sign in")
 home_page_pw.open_login_page_pw()
 login_page_pw.login_as_admin_pw()

 print("[PW][STEP] Clear cart before adding a product")
 product_page_pw.clear_cart_pw()

 print("[PW][STEP] Add one product and validate cart item")
 product_page_pw.open_bikes_main_link_pw()\
 .open_mountain_bikes_pw()\
 .add_first_product_to_cart_and_verify_pw()
 print("[PW][DONE] Basket flow completed")
```



## File: playwright\_tests\test\_header\_page\_playwright.py

```
import allure

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
def test_header_page_playwright(home_page_pw, login_page_pw, header_page_pw):
 print("\n[STEP] Open login page and sign in.")
 home_page_pw.open_login_page_pw()
 login_page_pw.login_as_admin_pw()

 print("[STEP] Validate header routes via page object helper.")
 header_page_pw.validate_header_navigation_pw()
 print("[DONE] Header navigation checks completed.")
```

## File: playwright\_tests\test\_home\_page\_playwright.py

```
import allure

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
def test_home_page_playwright(home_page_pw):
 print("\n[STEP] Open home page.")
 home_page_pw.open_home_page_pw()

 print("[STEP] Validate page title.")
 title = home_page_pw.get_title()
 print(f"[DEBUG] Retrieved title: '{title}'")

 print("[ASSERT] Title should contain 'Login'.")
 assert "Login" in title
 print("[DONE] Home page title validation passed.")
```

## File: playwright\_tests\test\_login\_page\_playwright.py

```
import allure
from playwright.sync_api import expect

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
def test_login_flow_pw(home_page_pw, login_page_pw):
 print("[STEP] Open login page")
 home_page_pw.open_login_page_pw()
 print("[STEP] Login as admin")
 login_page_pw.login_as_admin_pw()
 print("[ASSERT] Verify authenticated favorites link is visible")
 expect(login_page_pw.page.locator(login_page_pw.AUTHENTICATED_FAVORITES_LINK).first).to_be_visible()
 print("[STEP] Logout")
 home_page_pw.click_logout_button_pw()
 print("[DONE] test_login_flow_pw completed")
```

## File: playwright\_tests\test\_registration\_page\_playwright.py

```
import allure
import pytest

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
@pytest.mark.parametrize("scenario_name", ["required", "invalid_format"])
def test_registration_form_validation_pw(page, registration_page_pw, scenario_name):
 print(f"\n[STEP] Running Playwright registration scenario: {scenario_name}")
 registration_page_pw.open_registration_url_pw()

 if scenario_name == "required":
 print("[STEP] Triggering required field validation messages.")
 registration_page_pw.trigger_required_errors_pw()
 print("[ASSERT] Verifying required field validation messages.")
 registration_page_pw.expect_required_errors_pw()
 else:
 print("[STEP] Reloading page and triggering invalid format validation messages.")
 page.reload()
 registration_page_pw.trigger_format_errors_pw(
 incorrect_name="Tester",
 incorrect_email="zly_email.pl",
 short_password="123",
)
 print("[ASSERT] Verifying invalid format validation messages.")
 registration_page_pw.expect_format_errors_pw()
 print(f"[DONE] Completed Playwright registration scenario: {scenario_name}")
```

## File: playwright\_tests\test\_search\_and\_sort\_products\_pw.py

```
import allure

@allure.parent_suite("Playwright Framework")
@allure.suite("E-shop Tests")
def test_search_product_exists_pw(home_page_pw, login_page_pw, product_page_pw):
 print("\n[STEP] Open login page and sign in as admin.")
 home_page_pw.open_login_page_pw()
 login_page_pw.login_as_admin_pw()

 print("[STEP] Open bikes landing page and validate visible categories.")
 product_page_pw.open_bikes_main_link_pw()
 product_page_pw.assert_expected_bike_categories_pw()

 print("[STEP] Open Mountain Bikes subcategory.")
 product_page_pw.open_mountain_bikes_pw()

 print("[STEP] Validate All/Discounted filter consistency.")
 product_page_pw.assert_filter_counts_consistent_pw()

 print("[STEP] Validate all sorting options.")
 product_page_pw.all_sorting_options_pw()
 print("[DONE] Playwright search/sort scenario completed.")
```

## File: selenium\_tests\conftest.py

```
import contextlib
import allure
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

Telerik eShop: navigate here before clearing so cookies/storage/CDP target this origin.
ESHOP_ENTRY_URL = "https://demos.telerik.com/kendo-ui/eshop/"
ESHOP_CDP_ORIGIN = "https://demos.telerik.com"

--- Pages Import ---
from pages.selenium.header_page_selenium import HeaderPage
from pages.selenium.home_page_selenium import HomePage
from pages.selenium.login_page_selenium import LoginPage
from pages.selenium.registration_page_selenium import RegistrationPage
from pages.selenium.products_page_selenium import ProductsPage

def _heavy_storage_and_cache_reset(driver):
 """IndexedDB + cache clear (slow). Call once per browser session for speed."""
 with contextlib.suppress(Exception):
 driver.get(ESHOP_ENTRY_URL)
 with contextlib.suppress(Exception):
 driver.execute_cdp_cmd(
 "Storage.clearDataForOrigin",
 {"origin": ESHOP_CDP_ORIGIN, "storageTypes": "all"},
)
 with contextlib.suppress(Exception):
 driver.execute_cdp_cmd("Network.clearBrowserCache", {})

def _light_browser_reset(driver):
 """Fast isolation between tests: same origin, cookies + web storage only."""
 with contextlib.suppress(Exception):
 driver.get(ESHOP_ENTRY_URL)
 with contextlib.suppress(Exception):
 driver.delete_all_cookies()
 with contextlib.suppress(Exception):
 driver.execute_script(
 "try { localStorage.clear(); } catch (e) {}"
 "try { sessionStorage.clear(); } catch (e) {}"
)

--- Driver factory ---
class DriverFactory:
 @staticmethod
 def get_driver(run_remote, options):
 if run_remote:
 # Docker/remote execution.
 executor_url = "http://localhost:4444/wd/hub"
 driver = webdriver.Remote(
 command_executor=executor_url,
 options=options
)
 else:
 # Local execution.
 driver = webdriver.Chrome(options=options)
 return driver

--- Main Selenium WebDriver fixture ---
@pytest.fixture(scope="session")
def driver(request):
 # 1. Read runtime options from CLI.
 remote_opt = request.config.getoption("--remote").lower() == "true"
 headless_opt = request.config.getoption("--headless").lower() == "true"

 # 2. Configure Chrome options.
 options = Options()

 if headless_opt:
 # Chrome 109+: prefer new headless (closer to headed behavior).
 options.add_argument("--headless=new")

 # Disable password/pop-up features that can interfere with automation.
 options.add_argument("--disable-features=PasswordLeakDetection,SafeBrowsing")
```

```

options.add_argument("--disable-save-password-bubble")
options.add_argument("--disable-notifications")
options.add_argument("--guest")
options.page_load_strategy = "eager"
options.add_argument("--window-size=1920,1080")
options.add_argument("--disable-renderer-backgrounding")
options.add_argument("--disable-backgrounding-occluded-windows")

Improve stability in containerized/virtualized Linux.
options.add_argument("--disable-gpu")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")

Hide automation extensions/flags.
options.add_experimental_option("excludeSwitches", ["enable-automation"])
options.add_experimental_option("useAutomationExtension", False)

Profile preferences.
prefs = {
 "credentials_enable_service": False,
 "profile.password_manager_enabled": False,
 "autofill.profile_enabled": False
}
options.add_experimental_option("prefs", prefs)

3. Initialize driver.
driver = DriverFactory.get_driver(run_remote=remote_opt, options=options)
driver.set_page_load_timeout(4)
Explicit waits only: mixing implicit + WebDriverWait inflates every poll with
implicit delay and causes hard-to-debug hangs (see BasePage._wait).
driver.implicitly_wait(0)
if not headless_opt:
 with contextlib.suppress(Exception):
 driver.maximize_window()

One heavy Chromium reset per session (major speed win vs per-test CDP).
_heavy_storage_and_cache_reset(driver)
setattr(driver, "_selenium_heavy_storage_cleared", True)

yield driver

4. Teardown.
driver.quit()

--- Per-test browser-state reset on shared driver ---
@pytest.fixture(autouse=True)
def reset_browser_state(driver):
 """
 Keep tests isolated while reusing one browser instance.

 Heavy CDP (IndexedDB + network cache) runs once per session in ``driver`` fixture.
 Between tests: fast navigation + cookies + session/local storage only.
 Safe for pytest-xdist: each worker has its own session-scoped driver.
 """
 if getattr(driver, "_selenium_heavy_storage_cleared", False):
 _light_browser_reset(driver)
 else:
 _heavy_storage_and_cache_reset(driver)
 setattr(driver, "_selenium_heavy_storage_cleared", True)
 yield

--- FIXTURE STRON Selenium (Page Objects) ---
@pytest.fixture
def registration_page_selenium(driver):
 return RegistrationPage(driver)

@pytest.fixture
def home_page_selenium(driver):
 return HomePage(driver)

@pytest.fixture
def login_page_selenium(driver):
 return LoginPage(driver)

@pytest.fixture
def header_page_selenium(driver):

```

```

 return HeaderPage(driver)

@pytest.fixture
def product_page_selenium(driver):
 return ProductsPage(driver)

--- Automatic Allure screenshots on test failure ---
@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
 outcome = yield
 rep = outcome.get_result()

 # Capture screenshot only when test fails in call phase.
 if rep.when == "call" and rep.failed:
 if "driver" in item.fixturenames:
 driver = item.funcargs.get("driver")
 if driver:
 try:
 allure.attach(
 driver.get_screenshot_as_png(),
 name="failure_screenshot_selenium",
 attachment_type=allure.attachment_type.PNG
)
 except Exception as e:
 print(f"\n[Allure] Failed to capture screenshot: {e}")

```



## File: selenium\_tests\test\_adding\_multiple\_items\_to\_basket.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_multiple_products_total_calculation(home_page_selenium, login_page_selenium, product_page_selenium):
 print("[STEP] Open login page and authenticate.")
 home_page_selenium.open_login_page()
 login_page_selenium.login_as_admin()

 print("[STEP] Clear cart before multi-product total verification.")
 # POM clear_cart() returns to the Bikes landing page when done
 product_page_selenium.clear_cart()
 # Hard check on /ShoppingCart so we never add products on top of a stale cart
 product_page_selenium.assert_shopping_cart_empty()

 print("[STEP] Open mountain bikes category.")
 product_page_selenium.open_mountain_bikes()

 print("[STEP] Add multiple products and validate computed cart total.")
 # count=3 keeps Selenium total-validation stable
 product_page_selenium.add_multiple_products_and_verify_total(count=3)
 print("[DONE] Multi-product total calculation flow completed.")
```

## File: selenium\_tests\test\_basket\_selenium.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_add_to_cart_flow(home_page_selenium, login_page_selenium, product_page_selenium):
 print("\n[STEP][Selenium][Basket] Open login page and authenticate.")
 home_page_selenium.open_login_page()
 login_page_selenium.login_as_admin()

 print("[STEP][Selenium][Basket] Clear cart before scenario.")
 product_page_selenium.clear_cart()

 print("[STEP][Selenium][Basket] Open mountain bikes and add first product.")
 product_page_selenium.open_mountain_bikes()
 product_page_selenium.add_first_product_to_cart_and_verify()
 print("[DONE][Selenium][Basket] Add-to-cart flow completed.")
```

## File: selenium\_tests\test\_header\_page\_selenium.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_header_page_selenium(home_page_selenium, login_page_selenium, header_page_selenium):
 print("[STEP] Open login page.")
 # 1. Open login page and sign in.
 home_page_selenium.open_login_page()
 print("[STEP] Login as admin user.")
 login_page_selenium.login_as_admin()
 print("[STEP] Validate header navigation routes.")
 # 2. Validate all header navigation paths.
 header_page_selenium.validate_header_navigation()
 print("[DONE] Header navigation validation completed.")
```

## File: selenium\_tests/test\_home\_page\_selenium.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_homepage_title(home_page_selenium, login_page_selenium):
 print("\n[STEP] Open login page and authenticate user")
 home_page_selenium.open_login_page()
 login_page_selenium.login_as_admin()

 print("[STEP] Wait for home page URL after login")
 home_page_selenium.wait_for_url(home_page_selenium.HOME_PAGE_URL)

 print("[ASSERT] Validate home page title")
 actual_title = home_page_selenium.get_title()
 expected_title = "Home Page - Web"
 assert expected_title in actual_title, (
 f"Expected title containing '{expected_title}', got '{actual_title}'"
)
 print("[DONE] Home page title test passed")
```

## File: selenium\_tests\test\_login\_page\_selenium.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_login_flow(home_page_selenium, login_page_selenium):
 # 1. Open the login page.
 print("[STEP] Opening home and login pages")
 home_page_selenium.open_home_page()
 home_page_selenium.open_login_page()
 # 2. Log in as admin and verify post-login state.
 print("[STEP] Logging in as admin")
 login_page_selenium.login_as_admin()
 current_url = home_page_selenium.driver.current_url
 print(f"[DEBUG] URL after login: {current_url}")
 print("[ASSERT] Verifying user is no longer on login page")
 assert "Account/Login" not in current_url, (
 f"Expected not to remain on login page after auth, got URL: {current_url}"
)
 print("[ASSERT] Verifying user is inside e-shop area")
 assert "eshop" in current_url, f"Expected to be inside e-shop after login, got URL: {current_url}"
 print("[DONE] Selenium login flow validated")
```

## File: selenium\_tests\test\_registration\_page\_selenium.py

```
import allure
import pytest

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
@pytest.mark.parametrize("scenario_name", ["required", "invalid_format"])
def test_registration_form_validation(driver, registration_page_selenium, scenario_name):
 print(f"\n[STEP][Selenium][Registration] Start scenario: {scenario_name}")
 registration_page_selenium.open_registration_url()

 if scenario_name == "required":
 print("[STEP] Trigger required-field validation errors")
 registration_page_selenium.trigger_required_errors()
 print("[ASSERT] Validate required-field error messages")
 registration_page_selenium.assert_required_errors()
 else:
 print("[STEP] Re-open registration page before invalid-format scenario")
 registration_page_selenium.open_registration_url()
 print("[STEP] Trigger invalid-format validation errors")
 registration_page_selenium.trigger_format_errors(
 incorrect_name="Tester",
 incorrect_email="zly_email.pl",
 short_password="123",
)
 print("[ASSERT] Validate invalid-format error messages")
 registration_page_selenium.assert_format_errors()
 print(f"[DONE][Selenium][Registration] Scenario passed: {scenario_name}")
```

## File: selenium\_tests\test\_search\_and\_sort\_products\_selenium.py

```
import allure

@allure.parent_suite("Selenium Framework")
@allure.suite("E-shop Tests")
def test_search_product_exists(home_page_selenium, login_page_selenium, product_page_selenium):
 print("\n[STEP] Open login page and authenticate.")
 home_page_selenium.open_login_page()
 login_page_selenium.login_as_admin()

 print("[STEP] Open bikes landing page and validate category blocks.")
 product_page_selenium.open_bikes_main_link()
 product_page_selenium.assert_expected_bike_categories(
 ["Mountain Bikes", "Road Bikes", "Touring Bikes"]
)

 print("[STEP] Open mountain bikes category.")
 product_page_selenium.open_mountain_bikes()
 product_page_selenium.wait_for_page_load(2)

 print("[ASSERT] Validate All and Discounted filter consistency.")
 product_page_selenium.assert_filter_counts_consistent()

 print("[ASSERT] Validate all sorting options.")
 product_page_selenium.all_sorting_options()
 print("[DONE] Search and sort flow completed.")
```

**File: selenium\_tests\\_\_init\_\_.py**



## File: utils\config.py

```
utilities/config.py

class Config:
 # URLs
 BASE_URL = "https://demos.telerik.com/kendo-ui/eshop"
 SELENIUM_HUB = "http://localhost:4444/wd/hub"

 # Demo credentials (see README)
 USER_EMAIL = "jaxons.danniels@company.com"
 USER_PASSWORD = "User1234"

 # Timeouts
 DEFAULT_TIMEOUT = 5
```

**File: utils\\_\_init\_\_.py**